

Assignment 3

Student name: *Daniele Mensorio*

Course: *Numerical methods for Data Analysis* – Professor: *Prof.essa Rosanna Campagna*
Due date: *June 17th, 2024*

Wine Dataset

the data file **WineData.mat** containing a data matrix X , $X \in \mathbb{R}^{13 \times 178}$, of chemical analysis data of wines derived from the same area of Italy but originating from three different cultivars. The original data file can be found on the UCI Machine Learning Repository. The attributes, corresponding to the rows of the data matrix X are concentrations/levels of the following substances:

$$x^j = \begin{bmatrix} \text{Alcohol} \\ \text{Malic acid} \\ \text{Ash} \\ \text{Alcalinity of ash} \\ \text{Magnesium} \\ \text{Total phenols} \\ \text{Flavonoids} \\ \text{Nonflavonoid phenols} \\ \text{Proanthocyanins} \\ \text{Color intensity} \\ \text{Hue} \\ \text{OD280/OD315 of diluted wines} \\ \text{Proline} \end{bmatrix} \in \mathbb{R}^{13}, \quad 1 \leq j \leq 178$$

PCA. The **Principal Component Analysis** (PCA) is a method to look the data in a low dimensional subspace where the spread of the data in their principal direction is maximized. After computing the SVD (Singular Value Decomposition) of the matrix X , since it is the best possible rank k approximation of the data:

$$X_c = UDV^T$$

where:

$U \in \mathbb{R}^{13 \times 178}$ is a matrix with orthonormal columns that contains the left singular vectors

$D \in \mathbb{R}^{178 \times 178}$ is a diagonal matrix that contains the singular values

$V \in \mathbb{R}^{16 \times 178}$ is a matrix with orthonormal columns that contain the right singular vectors

Since the matrix X dataset contains a lot information, the original data can be represented by only a few set of information that allow a big reduction useful both for

computational effort and also for eliminate all the noise and capture only the essential information from the column vectors.

After computing the first k principal component of the data,

$$x_c^{(j)} = U_k D_k V_k^T = U_k z^{(j)}$$

$$z^{(j)} = D_k V_k^T e_j$$

$$z^{(j)} = u_k^T x^{(j)}$$

$$Z = U_k^T X$$

Let's visualize the scatter-plot of one component, that correspond to a row of the Z matrix, against the other component.

The use of PCA (Principal Component Analysis) as visualisation tool is very interesting and the reason is that PCA allow to visualise centered data in two- or three-dimensional scatter-plot with a the best possible spread. That's because it can be proven that the projection direction that maximizes the spread of the data is given by the first left singular vector, $q = u(1)$:

$$\max\{\text{spread}(y)\} = \sigma_1 = \|\mathbf{X}^T \mathbf{u}^{(1)}\|$$

The other projection direction, orthogonal to u_1 , that gives the second largest spread is $u^{(2)}$:

$$\max\{\text{spread}(\tilde{y})\} = \sigma_2 = \|\tilde{\mathbf{X}}^T \mathbf{u}^{(2)}\|$$

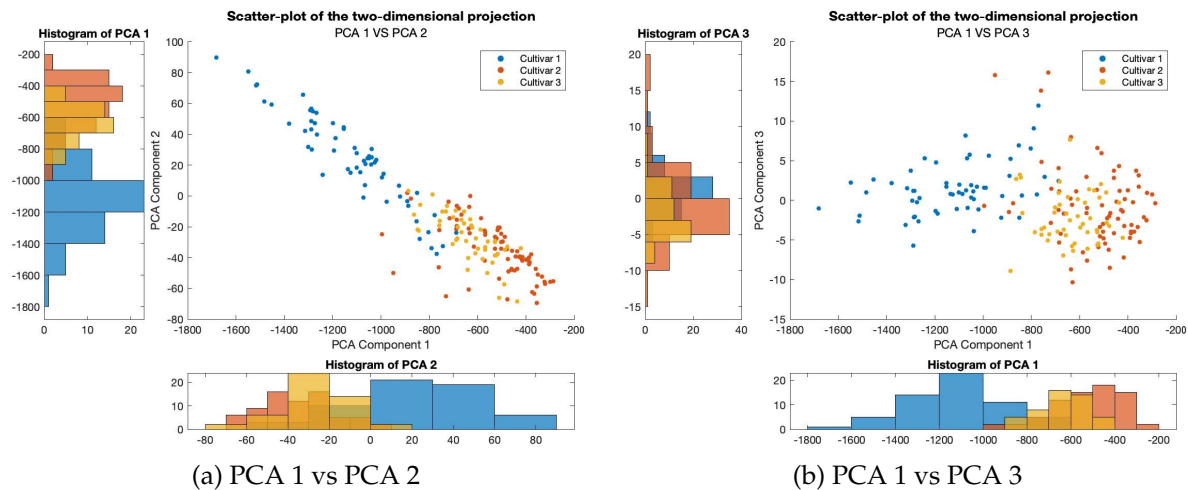
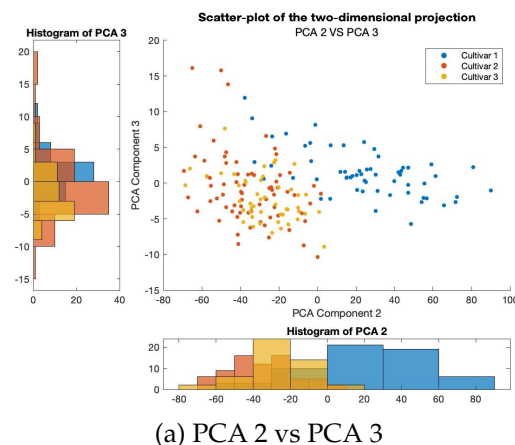


Figure 1: a) Scatter-plot of the projection of the first principal component against the second, with the histograms of the two represented components. b) Scatter-plot of the projection of the first principal component against the third, with the histograms of the two represented components. c) Scatter-plot of the projection of the second principal component against the third, with the histograms of the two represented components..



LDA. Linear Discriminant Analysis is a method for looking the data into a low dimensional subspace but into directions that minimizes the spread of each cluster and, simultaneously, maximizes the separation between each cluster.

Let's consider a data matrix X , $X \in \mathbb{R}^{np}$, the aim of LDA is to find the direction vector, $q \in \mathbb{R}^p$ with $\|q\| = 1$, that maximizes the between-cluster scatter matrix of $\bar{X}_c$, S_b , and that minimizes the within-cluster scatter matrix of the within-cluster centered data matrix X_w , S_w . So, the problem can be stated in this way: the goal is to find the projection direction q :

$$q = \arg \max \{q^T S_b q\} \quad \text{constrained to} \quad q^T S_w q = 1$$

where:

•

$$S_w = \sum_{l=1}^k S_l$$

$$S_l = X_{l,c} X_{l,c}^T = \sum_{j \in I_l} (x^{(j)} - c^{(l)})(x^{(j)} - c^{(l)})^T$$

•

$$S_b = \bar{X}_c \bar{X}_c^T = \sum_{l=1}^k p_l (c^{(l)} - c)(c^{(l)} - c)^T = \sum_{l=1}^k \sum_{j=1}^{p_l} (c^{(l)} - c)(c^{(l)} - c)^T$$

To find the solution of this maximization problem with the constraint it is possible to use the Lagrange Multiplier method. So, the solution of the Lagrange function, $L_\lambda(q)$,

$$L_\lambda(q) = q^T S_b q - \lambda(q^T S_w q - 1)$$

where $\lambda \in \mathbb{R}$ is the Lagrange Multiplier, that satisfies that its gradient with respect to q is equal to 0,

$$\nabla_q L_\lambda(q) = 2(S_b q - \lambda S_w q) = 0$$

are also the solution of the generalized eigenvalue problem:

$$S_b q = \lambda S_w q$$

if we consider that S_w is full of rank, so it is invertible, the eigenvalue problem became:

$$S_w^{-1} S_b q = \lambda q$$

So, q is the eigenvector associated with eigenvalue λ of the matrix $S_w^{-1}S_b$. Since, we want that the eigenvalue of the matrix are real, non-negative and arranged in a decreasing order, using the eigenvalue theorem, it is ideal to search for a symmetric, non-definite matrix: that matrix is as follow:

$$S_b^{\frac{1}{2}}S_w^{-1}S_b^{\frac{1}{2}}S_b^{\frac{1}{2}}q = \lambda S_b^{\frac{1}{2}}q$$

or

$$S_b^{\frac{1}{2}}S_w^{-1}S_b^{\frac{1}{2}}w = \lambda w$$

where $w = S_b^{\frac{1}{2}}q$.

Then, after a substitution in the objection function, this latest of the the j -th projection direction q^j is equal to the j -th eigenvalue, λ_j . For the maximum projection direction, $j = 1$. This implies that the optimal direction is that of the eigenvector corresponding to the largest eigenvalue of the matrix $S_w^{-1}S_b$.

For unsure that the matrix S_w is invertible, we apply a regularization:

$$S_w = S_w + \epsilon I_n = U\Lambda_\epsilon U^T$$

where the diagonal entries of Λ_ϵ are the squared singular values of X_w plus the regularization, $d_j^2 + \epsilon$ and $\epsilon > 0$ is a small regularization parameter (rule of thumb: $\epsilon = \tau d_j^2$, where $\tau > 0$ is a small parameter, usually $\tau = 10^{-10}$).

To unsure, also, that the eigenvalues are, also in reality, not only theoretically, real and positive values, we apply the Cholesky Factorization where the symmetric and positive definite matrix of S_w is:

$$A = K^{-T}S_bK^{-1}$$

After all this algebraic computation, we find the row vector of size p , y , that collects the projection of the column vector of X , $x^{(j)}$, on the direction vector, q :

$$y = q^T X$$

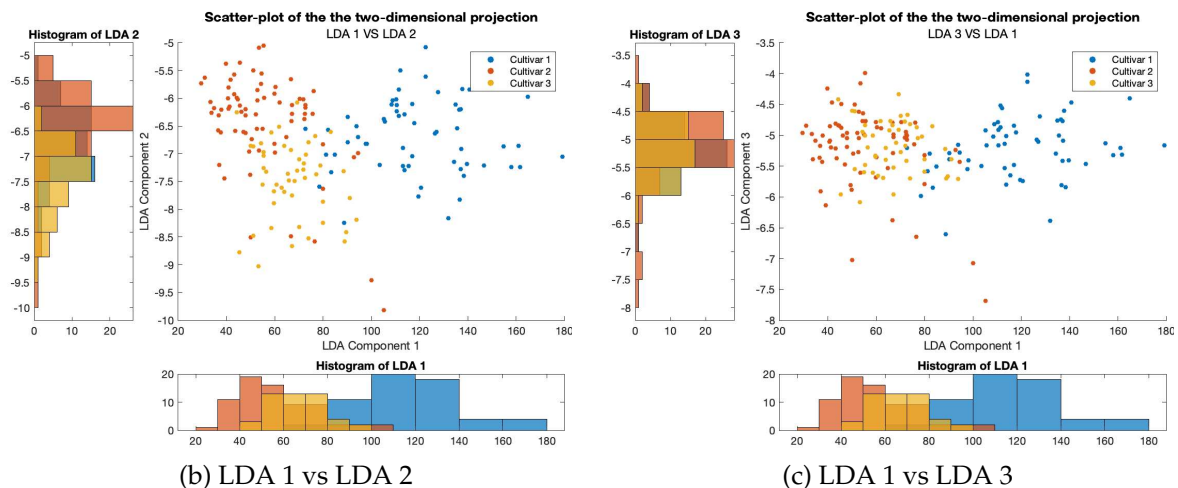
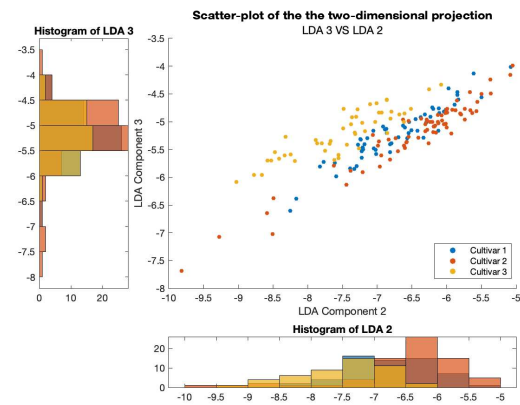


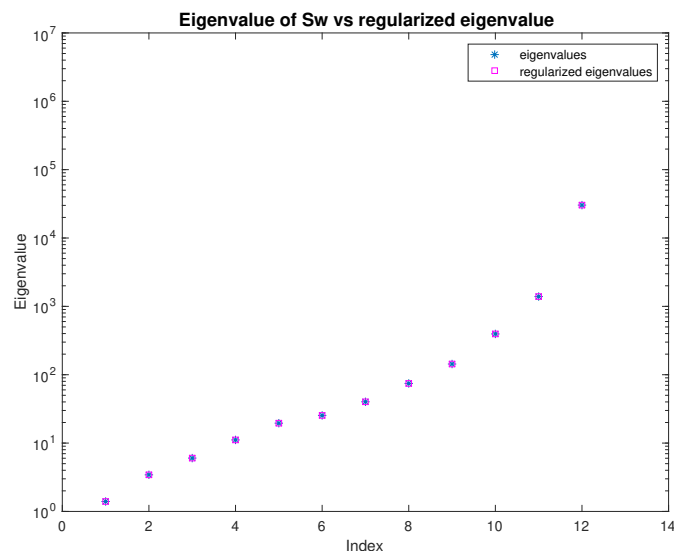
Figure 2: a) Scatter-plot of the projection of the first LDA component against the second, with the histograms of the two represented components. b) Scatter-plot of the projection of the first LDA component against the third, with the histograms of the two represented components. c) Scatter-plot of the projection of the second LDA component against the third, with the histograms of the two represented components.



(a) LDA 2 vs LDA 3

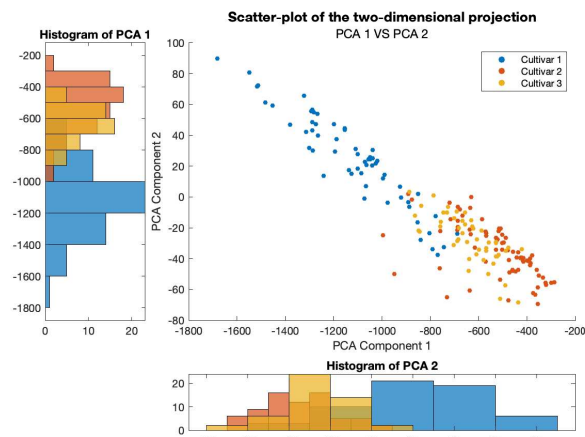
In figure 4a, it is possible to see the difference between the progression of the eigenvalue over indexes and the respective regularized eigenvalue.

Figure 3: Eigenvalue progression over iteration versus the regularized eigenvalue progression over iteration.

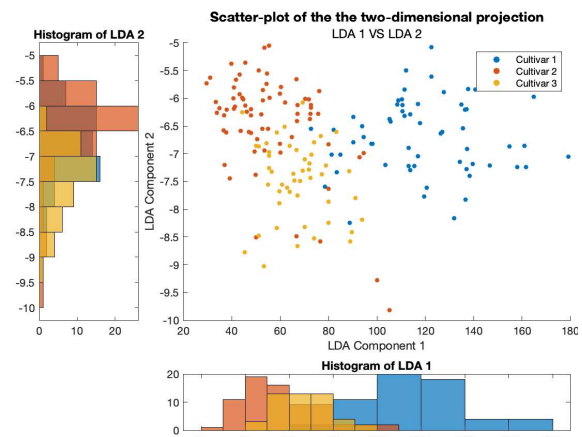


(a) Eigenvalue progression

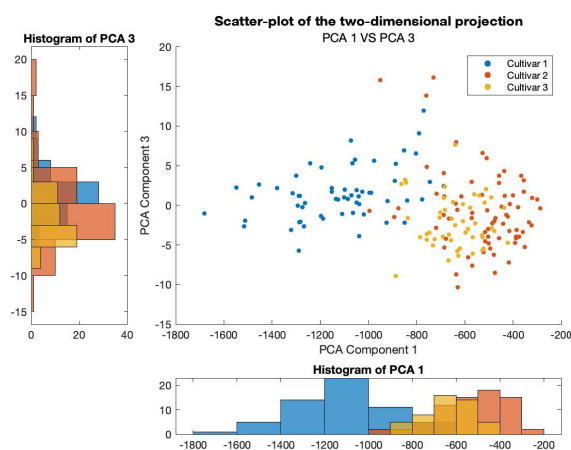
0.1. PCA VS LDA. Difference between PCA versus LDA method: while the **Principal Component Analysis (PCA)** is a method to look the data in a low dimensional subspace where the spread of the data in their principal direction is maximized; **Linear Discriminant Analysis** is a method for looking the data into a low dimensional subspace but into directions that minimizes the spread of each cluster and, simultaneously, maximizes the separation between each cluster. That's if data suggest cluster, PCA doesn't guarantee that the clusters are visible in the projection of the data onto the direction.



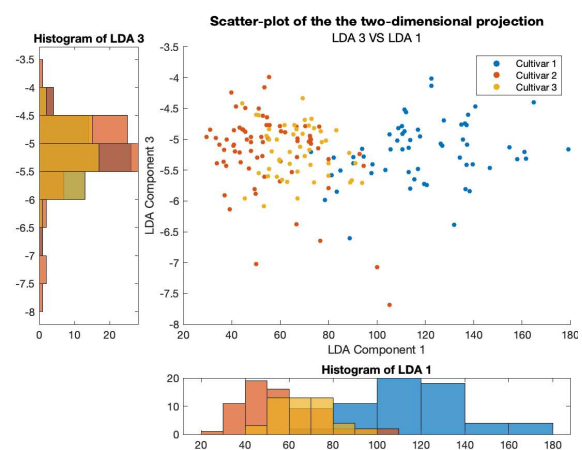
(b) PCA 1 vs PCA 2



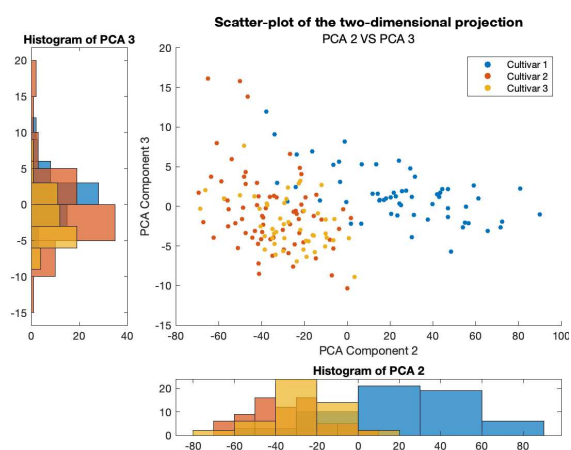
(c) LDA 1 vs LDA 2



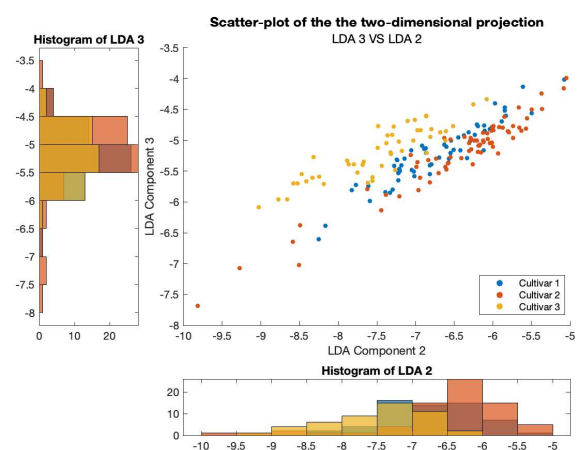
(d) PCA 1 vs PCA 3



(e) LDA 1 vs LDA 3



(f) PCA 2 vs PCA 3



(g) LDA 2 vs LDA 3

Hand Written Digit Dataset

The **HandWrittenDigits.mat** dataset contain the data matrix X and the data matrix I . The data X consist of black and white pixel images of handwritten digits collected from US postal envelopes. An image, strongly down-sampled, consists of 16×16 pixels, with gray scale values that are scaled into the interval $[0, 1]$, the value 1 representing white, and 0 black. The pixel values are listed in lexicographical order and stacked into a vector of length $n = (16)^2 = 256$. Our sample contains $p = 1707$ images, hence the data matrix is $X \in \mathbb{R}^{256 \times 1707}$.

The vector $I \in \mathbb{N}^{1707}$ contains a list of indices, or attributes, from 0 to 9, indicating that

$$x^{(j)} \text{ represents the number } I(j) \in \{0, 1, \dots, 9\}$$

PCA. The Principal Component Analysis (PCA) is a method to look the data in a low dimensional subspace where the spread of the data in their principal direction is maximized. After computing the SVD (Singular Value Decomposition) of the matrix X , since it is the best possible rank k approximation of the data:

$$X_c = UDV^T$$

where:

$U \in \mathbb{R}^{256 \times 1707}$ is a matrix with orthonormal columns that contains the left singular vectors

$D \in \mathbb{R}^{1707 \times 1707}$ is a diagonal matrix that contains the singular values

$V \in \mathbb{R}^{256 \times 1707}$ is a matrix with orthonormal columns that contain the right singular vectors

Since the matrix X dataset contains a lot information, the original data can be represented by only a few set of information that allow a big reduction useful both for computational effort and also for eliminate all the noise and capture only the essential information from the column vectors.

After computing the first k principal component of the data,

$$x_c^{(j)} = U_k D_k V_k^T = U_k z^{(j)}$$

$$z^{(j)} = D_k V_k^T e_j$$

$$z^{(j)} = u_k^T x^{(j)}$$

$$Z = U_k^T X$$

Let's visualize the scatter-plot of one component, that correspond to a row of the Z matrix, against the other component.

The use of PCA (Principal Component Analysis) as visualisation tool is very interesting and the reason is that PCA allow to visualise centered data in two- or three-dimensional scatter-plot with a the best possible spread. That's because it can be

proven that the projection direction that maximizes the spread of the data is given by the first left singular vector, $q = u(1)$:

$$\max\{\text{spread}(y)\} = \sigma_1 = \|\mathbf{X}^T \mathbf{u}^{(1)}\|$$

The other projection direction, orthogonal to u_1 , that gives the second largest spread is $u^{(2)}$:

$$\max\{\text{spread}(\tilde{y})\} = \sigma_2 = \|\tilde{\mathbf{X}}^T \mathbf{u}^{(2)}\|$$

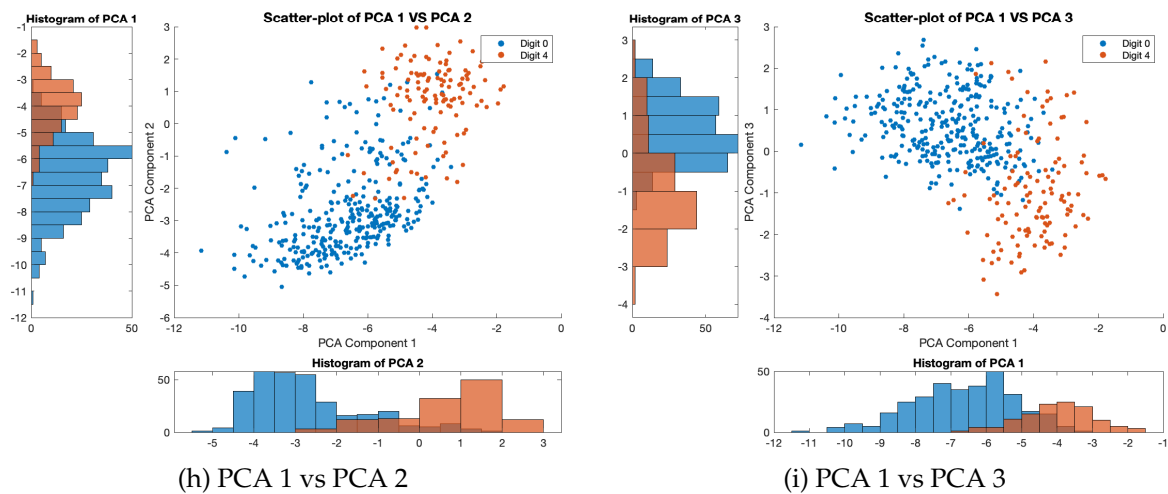
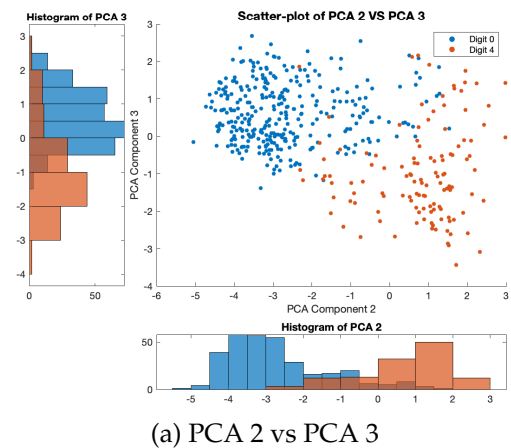


Figure 4: a) Scatter-plot of the projection of the first principal component against the second, with the histograms of the two represented components. b) Scatter-plot of the projection of the first principal component against the third, with the histograms of the two represented components. c) Scatter-plot of the projection of the second principal component against the third, with the histograms of the two represented components..



LDA. Linear Discriminant Analysis is a method for looking the data into a low dimensional subspace but into directions that minimizes the spread of each cluster and, simultaneously, maximizes the separation between each cluster.

Let's consider a data matrix X , $X \in \mathbb{R}^{np}$, the aim of LDA is to find the direction vector, $q \in \mathbb{R}^p$ with $\|q\| = 1$, that maximizes the between-cluster scatter matrix of $\bar{X}_c$, S_b , and that minimizes the within-cluster scatter-matrix of the within-cluster centered data matrix X_w , S_w . So, the problem can be stated in this way: the goal is to find the

projection direction q :

$$q = \arg \max \{q^T S_b q\} \quad \text{constrained to} \quad q^T S_w q = 1$$

where:

•

$$S_w = \sum_{l=1}^k S_l$$

$$S_l = X_{l,c} X_{l,c}^T = \sum_{j \in I_l} (x^{(j)} - c^{(l)})(x^{(j)} - c^{(l)})^T$$

•

$$S_b = \bar{X}_c \bar{X}_c^T = \sum_{l=1}^k p_l (c^{(l)} - c)(c^{(l)} - c)^T = \sum_{l=1}^k \sum_{j=1}^{p_l} (c^{(l)} - c)(c^{(l)} - c)^T$$

To find the solution of this maximization problem with the constraint it is possible to use the Lagrange Multiplier method. So, the solution of the Lagrange function, $L_\lambda(q)$,

$$L_\lambda(q) = q^T S_b q - \lambda(q^T S_w q - 1)$$

where $\lambda \in \mathbb{R}$ is the Lagrange Multiplier, that satisfies that its gradient with respect to q is equal to 0,

$$\nabla_q L_\lambda(q) = 2(S_b q - \lambda S_w q) = 0$$

are also the solution of the generalized eigenvalue problem:

$$S_b q = \lambda S_w q$$

if we consider that S_w is full of rank, so it is invertible, the eigenvalue problem became:

$$S_w^{-1} S_b q = \lambda q$$

So, q is the eigenvector associated with eigenvalue λ of the matrix $S_w^{-1} S_b$. Since, we want that the eigenvalue of the matrix are real, non-negative and arranged in a decreasing order, using the eigenvalue theorem, it is ideal to search for a symmetric, non-definite matrix: that matrix is as follow:

$$S_b^{\frac{1}{2}} S_w^{-1} S_b^{\frac{1}{2}} q = \lambda S_b^{\frac{1}{2}} q$$

or

$$S_b^{\frac{1}{2}} S_w^{-1} S_b^{\frac{1}{2}} w = \lambda w$$

where $w = S_b^{\frac{1}{2}} q$.

Then, after a substitution in the objection function, this latest of the the j -th projection direction q^j is equal to the j -th eigenvalue, λ_j . For the maximum projection direction, $j = 1$. This implies that the optimal direction is that of the eigenvector corresponding to the largest eigenvalue of the matrix $S_w^{-1} S_b$.

For unsure that the matrix S_w is invertible, we apply a regularization:

$$S_w = S_w + \epsilon I_n = U \Lambda_\epsilon U^T$$

where the diagonal entries of Λ_ϵ are the squared singular values of X_w plus the regularization, $d_j^2 + \epsilon$ and $\epsilon > 0$ is a small regularization parameter (rule of thumb: $\epsilon = \tau d_j^2$, where $\tau > 0$ is a small parameter, usually $\tau = 10^{-10}$).

To ensure, also, that the eigenvalues are, also in reality, not only theoretically, real and positive values, we apply the Cholesky Factorization where the symmetric and positive definite matrix of S_w is:

$$A = K^{-T} S_b K^{-1}$$

After all this algebraic computation, we find the row vector of size p , y , that collects the projection of the column vector of X , $x^{(j)}$, on the direction vector, q :

$$y = q^T X$$

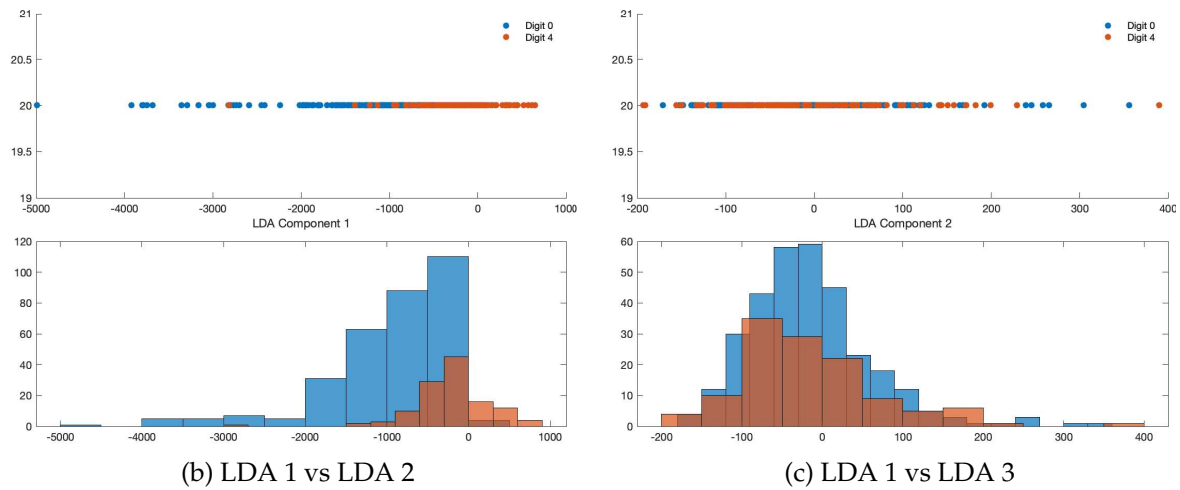
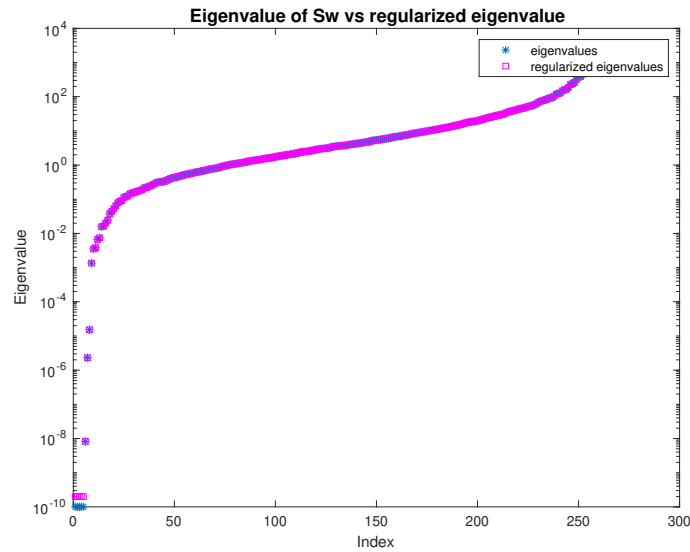


Figure 5: a) Scatter-plot of the projection of the first LDA component with the histograms of the represented component. b) Scatter-plot of the projection of the second LDA component, with the histograms of the represented component.

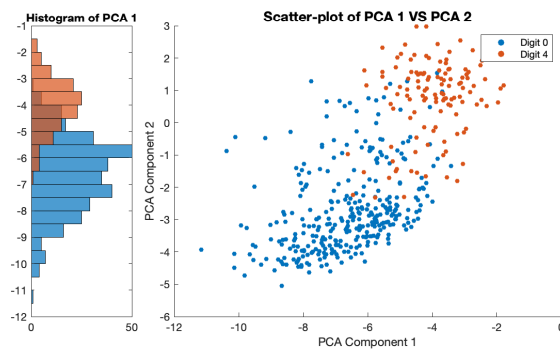
In figure 7a, it is possible to see the difference between the progression of the eigenvalue over indexes and the respective regularized eigenvalue.

Figure 6: Eigenvalue progression over iteration versus the regularized eigenvalue progression over iteration.

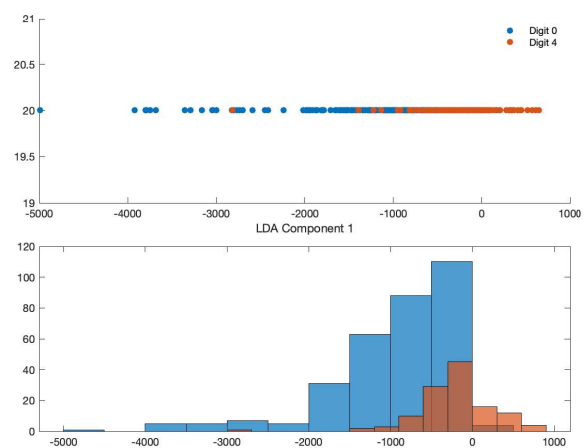


(a) Eigenvalue progression

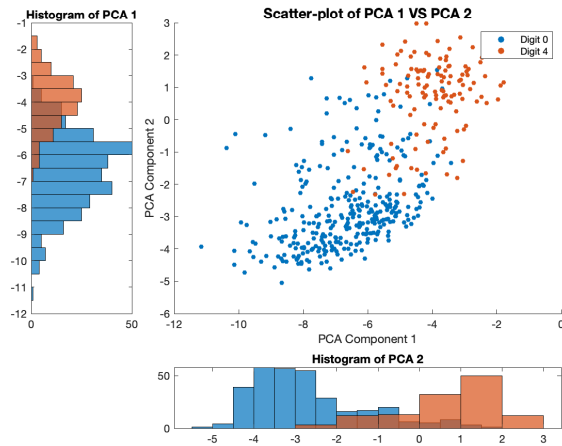
PCA VS LDA. Difference between PCA versus LDA method: while the **Principal Component Analysis** (PCA) is a method to look the data in a low dimensional subspace where the spread of the data in their principal direction is maximized; **Linear Discriminant Analysis** is a method for looking the data into a low dimensional subspace but into directions that minimizes the spread of each cluster and, simultaneously, maximizes the separation between each cluster. That's if data suggest cluster, PCA doesn't guarantee that the clusters are visible in the projection of the data onto the direction.



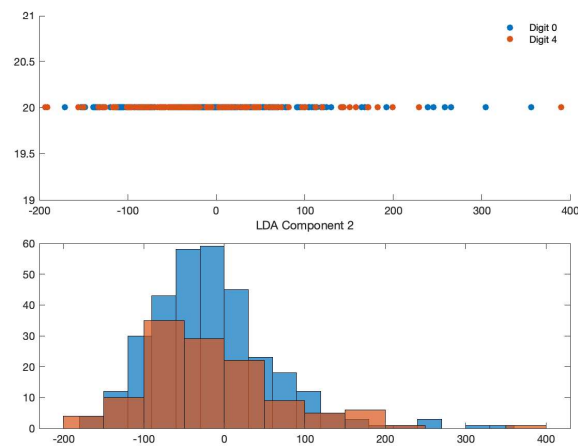
(b) PCA 1 vs PCA 2



(c) LDA 1



(d) PCA 1 vs PCA 3



(e) LDA 2

NMF. The rank k **Non-negative matrix Factorization** (NMF) of the matrix X , $X \in \mathbb{R}^{n \times p}$, is used even if SVD is the best possible rank k approximation of the same matrix, the preceding method don't ensure that the feature vector are non-negative. If we consider the case in which we have a grey-scale image with pixel value and the feature vectors are not non-negative values, they're not a real representation of the feature vector in the data: it is not possible a consideration, based on reality, of the implication of the grey-scale image with pixel value. So, rank- k NMF is a tool that work with non-negative matrix to find a rank- k approximation of the original matrix constrained to the fact that the entries of all matrices are non-negative.

We use the **Alternating Non-negative Least Squares** (ANLS) algorithm:

1. Given X , a data matrix with non-negative entries, $X \in \mathbb{R}_+^{n \times p}$, the rank $k < \min(n, p)$ of the approximation, a tolerance $\tau > 0$, and $tmax$, the maximum number of iterations.
2. **Initialization step:** $W = W_0 \in \mathbb{R}^n$ with columns scaled to have unit ∞ -norm. Set the iteration (t , the counting is starting), $t = 0$.
3. **Iteration Step:**

- **First Updating step:** Solve the first least square problem in order to find H^t , using W^{t-1} (that, for the first step, is $W^{t-1} = W^0$):

$$H^t = \arg \min \|X - WH\|_F \quad \text{subject to} \quad H_{ij} \geq 0 \quad \text{or} \quad H \in \mathbb{R}_+^{k \times p}$$

- **Second Updating Step:** Solve the second least square problem in order to find W^t , using H^t :

$$W^t = \arg \min \|X^T - W^T H^T\|_F \quad \text{subject to} \quad HW_{ij} \geq 0 \quad \text{or} \quad HW \in \mathbb{R}_+^{n \times k}$$

- **Scaling step:** Scaling the matrix W^t to ensure that it is made up of non-negative entries. It is use the scaling matrix, $\tilde{W}$:

$$\tilde{W} = WL^{-1} = \left[\frac{w^{(1)}}{\lambda_1} \frac{w^{(2)}}{\lambda_2} \dots \frac{w^{(k)}}{\lambda_k} \right]$$

where

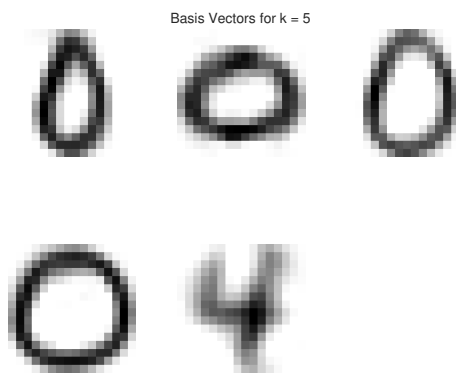
$$\lambda_j = \|w^{(j)}\|_\infty = \max_{1 \leq l \leq n} w_l^{(j)}$$

such that the entries have a range from 0 to 1. And set $W^t = \tilde{W}^t$

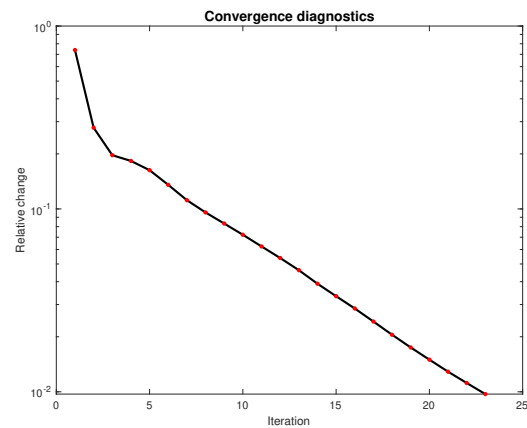
4. **Convergence step:** Check the convergence: When $t1$ (after the first iteration, at the second iteration), **if** condition,

$$\frac{\|W^{t+1} - W^t\|_F}{\|W^t\|_F} + \frac{\|H^{t+1} - H^t\|_F}{\|H^t\|_F}$$

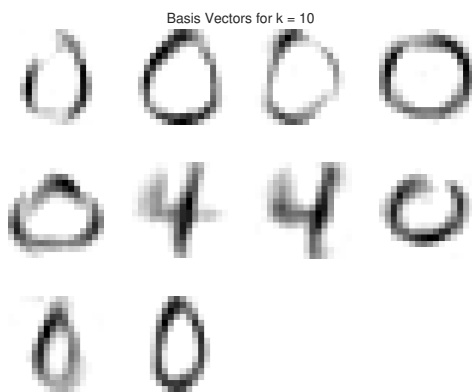
is satisfied or $t = tmax$ (the maximum fixed iteration is reached), **stop**, **else** increase $t = t + 1$ and continue from step 3.



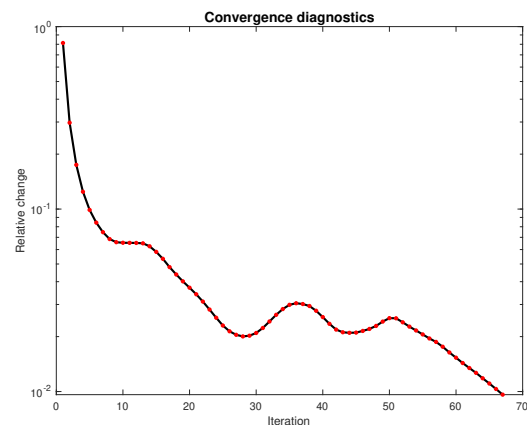
(f) feature vector $k = 5$



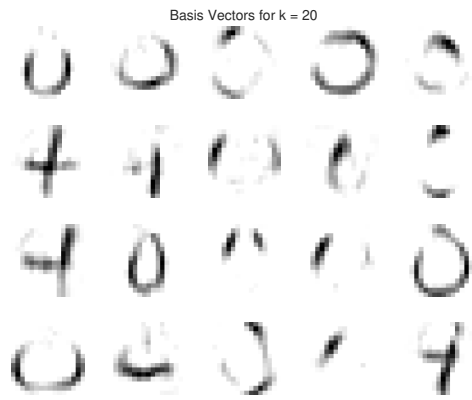
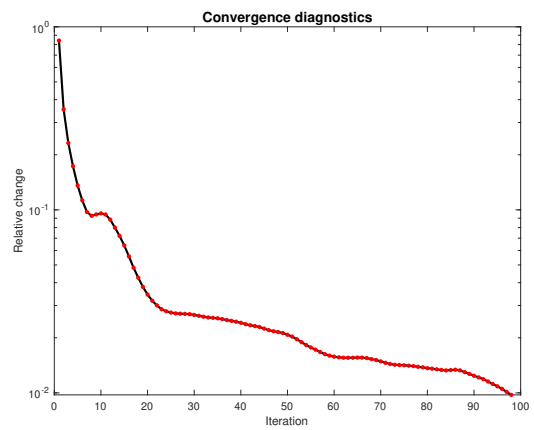
(g) stopping criterion for $k = 5$



(h) feature vector $k = 10$



(i) stopping criterion for $k = 10$

(j) feature vector $k = 20$ (k) stopping criterion for $k = 20$

Iris Dataset

The data file **IrisDataAnnotated.mat** consists of a data matrices X and I . The matrix X consists of 150 vectors, $X \in \mathbb{R}^{4 \times 150}$, each one having four components. The data corresponds to measurements of certain dimensions in three species of flowers, *Iris setosa*, *Iris versicolor*, and *Iris virginica*, and the components of the data vectors have the following attributes:

$$x^j = \begin{bmatrix} \text{sepal length in cm} \\ \text{sepal width in cm} \\ \text{petal length in cm} \\ \text{petal width in cm} \end{bmatrix} \in \mathbb{R}^4, \quad 1 \leq j \leq 150$$

The data matrix I , $I \in \mathbb{R}^{1 \times 150}$, is the label matrix in which to each column is assigned an iris species between the three of the flowers:

$$1 = \text{Iris Setosa} \quad 2 = \text{Iris Versicolor} \quad 3 = \text{Iris Virginica}$$

PCA. The Principal Component Analysis (PCA) is a method to look the data in a low dimensional subspace where the spread of the data in their principal direction is maximized. After computing the SVD (Singular Value Decomposition) of the matrix X , since it is the best possible rank k approximation of the data:

$$X_c = UDV^T$$

where:

$U \in \mathbb{R}^{4 \times 150}$ is a matrix with orthonormal columns that contains the left singular vectors

$D \in \mathbb{R}^{150 \times 150}$ is a diagonal matrix that contains the singular values

$V \in \mathbb{R}^{4 \times 150}$ is a matrix with orthonormal columns that contain the right singular vectors

Since the matrix X dataset contains a lot information, the original data can be represented by only a few set of information that allow a big reduction useful both for computational effort and also for eliminate all the noise and capture only the essential information from the column vectors.

After computing the first k principal component of the data,

$$x_c^{(j)} = U_k D_k V_k^T = U_k z^{(j)}$$

$$z^{(j)} = D_k V_k^T e_j$$

$$z^{(j)} = u_k^T x^{(j)}$$

$$Z = U_k^T X$$

Let's visualize the scatter-plot of one component, that correspond to a row of the Z matrix, against the other component.

The use of PCA (Principal Component Analysis) as visualisation tool is very interesting and the reason is that PCA allow to visualise centered data in two- or three-dimensional scatter-plot with a the best possible spread. That's because it can be

proven that the projection direction that maximizes the spread of the data is given by the first left singular vector, $q = u(1)$:

$$\max\{\text{spread}(y)\} = \sigma_1 = \|\mathbf{X}^T \mathbf{u}^{(1)}\|$$

The other projection direction, orthogonal to u_1 , that gives the second largest spread is $u^{(2)}$:

$$\max\{\text{spread}(\tilde{y})\} = \sigma_2 = \|\tilde{\mathbf{X}}^T \mathbf{u}^{(2)}\|$$

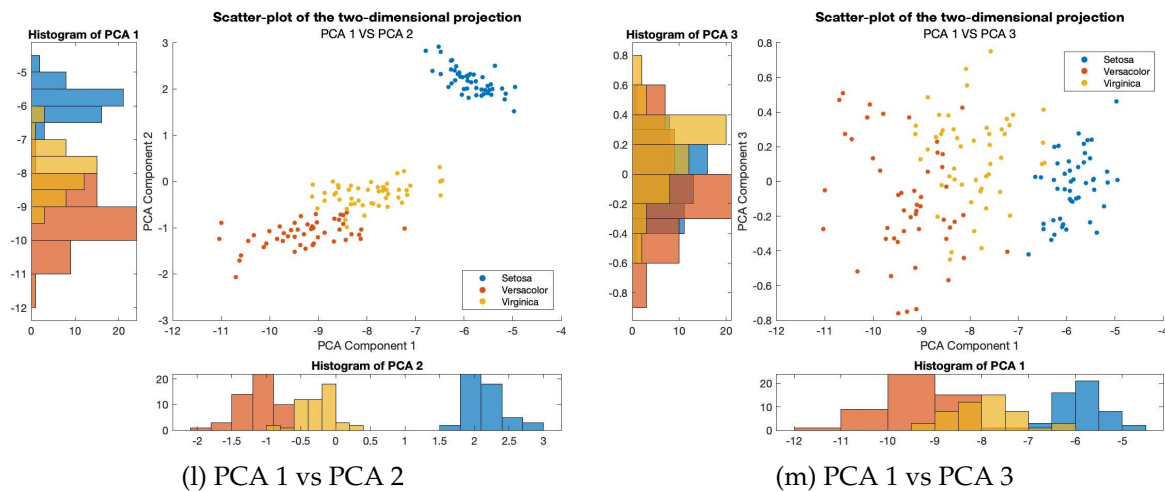
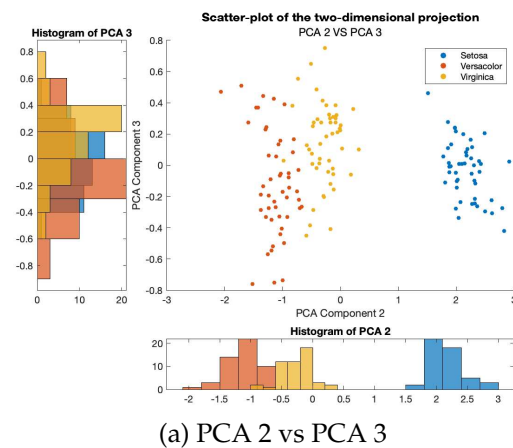


Figure 7: a) Scatter-plot of the projection of the first principal component against the second, with the histograms of the two represented components. b) Scatter-plot of the projection of the first principal component against the third, with the histograms of the two represented components. c) Scatter-plot of the projection of the second principal component against the third, with the histograms of the two represented components..



LDA. Difference between PCA versus LDA method: while the **Principal Component Analysis** (PCA) is a method to look the data in a low dimensional subspace where the spread of the data in their principal direction is maximized; **Linear Discriminant Analysis** is a method for looking the data into a low dimensional subspace but into directions that minimizes the spread of each cluster and, simultaneously, maximizes the separation between each cluster. That's if data suggest cluster, PCA doesn't guarantee that the clusters are visible in the projection of the data onto the direction.

Linear Discriminant Analysis is a method for looking the data into a low dimensional subspace but into directions that minimizes the spread of each cluster and, simultaneously, maximizes the separation between each cluster.

Let's consider a data matrix X , $X \in \mathbb{R}^{n \times p}$, the aim of LDA is to find the direction vector, $q \in \mathbb{R}^p$ with $\|q\| = 1$, that maximizes the between-cluster scatter matrix of $\bar{X}_c$, S_b , and that minimizes the within-cluster scatter-matrix of the within-cluster centered data matrix X_w , S_w . So, the problem can be stated in this way: the goal is to find the projection direction q :

$$q = \arg \max \{q^T S_b q\} \quad \text{constrained to} \quad q^T S_w q = 1$$

where:

•

$$S_w = \sum_{l=1}^k S_l$$

$$S_l = X_{l,c} X_{l,c}^T = \sum_{j \in I_l} (x^{(j)} - c^{(l)})(x^{(j)} - c^{(l)})^T$$

•

$$S_b = \bar{X}_c \bar{X}_c^T = \sum_{l=1}^k p_l (c^{(l)} - c)(c^{(l)} - c)^T = \sum_{l=1}^k \sum_{j=1}^{p_l} (c^{(l)} - c)(c^{(l)} - c)^T$$

To find the solution of this maximization problem with the constraint it is possible to use the Lagrange Multiplier method. So, the solution of the Lagrange function, $L_\lambda(q)$,

$$L_\lambda(q) = q^T S_b q - \lambda(q^T S_w q - 1)$$

where $\lambda \in \mathbb{R}$ is the Lagrange Multiplier, that satisfies that its gradient with respect to q is equal to 0,

$$\nabla_q L_\lambda(q) = 2(S_b q - \lambda S_w q) = 0$$

are also the solution of the generalized eigenvalue problem:

$$S_b q = \lambda S_w q$$

if we consider that S_w is full of rank, so it is invertible, the eigenvalue problem became:

$$S_w^{-1} S_b q = \lambda q$$

So, q is the eigenvector associated with eigenvalue λ of the matrix $S_w^{-1} S_b$. Since, we want that the eigenvalue of the matrix are real, non-negative and arranged in a decreasing order, using the eigenvalue theorem, it is ideal to search for a symmetric, non-definite matrix: that matrix is as follow:

$$S_b^{\frac{1}{2}} S_w^{-1} S_b^{\frac{1}{2}} q = \lambda S_b^{\frac{1}{2}} q$$

or

$$S_b^{\frac{1}{2}} S_w^{-1} S_b^{\frac{1}{2}} w = \lambda w$$

where $w = S_b^{\frac{1}{2}} q$.

Then, after a substitution in the objection function, this latest of the the j -th projection

direction q^j is equal to the j -th eigenvalue, λ_j . For the maximum projection direction, $j = 1$. This implies that the optimal direction is that of the eigenvector corresponding to the largest eigenvalue of the matrix $S_w^{-1}S_b$.

For unsure that the matrix S_w is invertible, we apply a regularization:

$$S_w = S_w + \epsilon I_n = U\Lambda_\epsilon U^T$$

where the diagonal entries of Λ_ϵ are the squared singular values of X_w plus the regularization, $d_j^2 + \epsilon$ and $\epsilon > 0$ is a small regularization parameter (rule of thumb: $\epsilon = \tau d_j^2$, where $\tau > 0$ is a small parameter, usually $\tau = 10^{-10}$).

To unsure, also, that the eigenvalues are, also in reality, not only theoretically, real and positive values, we apply the Cholesky Factorization where the symmetric and positive definite matrix of S_w is:

$$A = K^{-T}S_bK^{-1}$$

After all this algebraic computation, we find the row vector of size p , y , that collects the projection of the column vector of X , $x^{(j)}$, on the direction vector, q_j :

$$y = q^T X$$

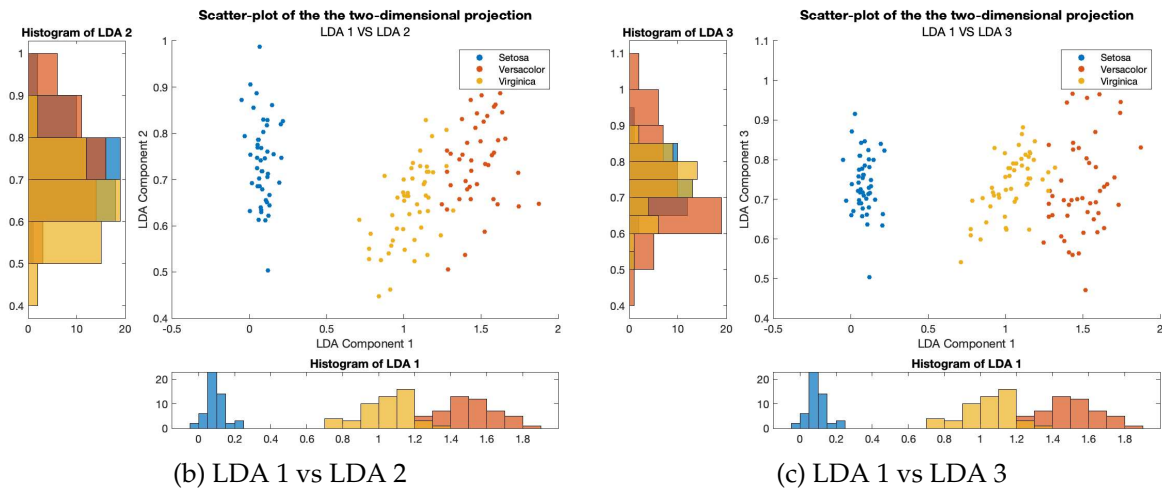
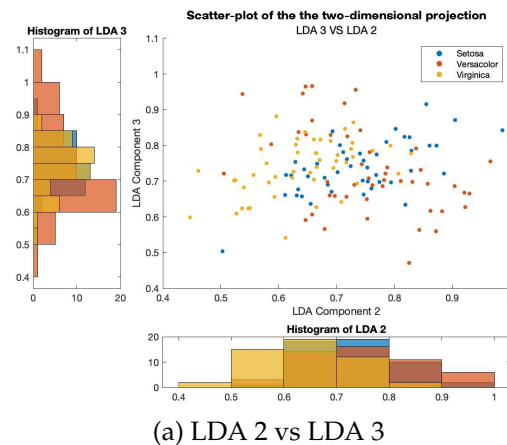
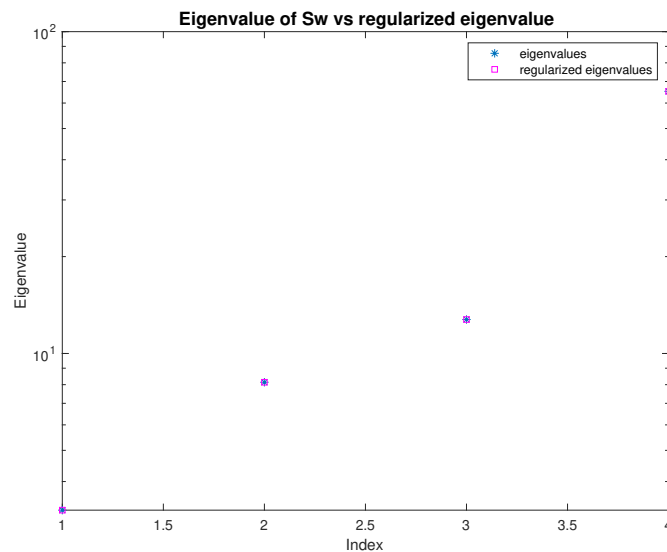


Figure 8: a) Scatter-plot of the projection of the first LDA component against the second, with the histograms of the two represented components. b) Scatter-plot of the projection of the first LDA component against the third, with the histograms of the two represented components. c) Scatter-plot of the projection of the second LDA component against the third, with the histograms of the two represented components.



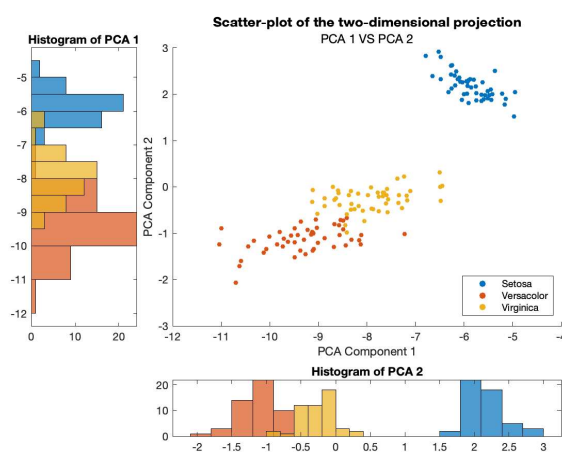
In figure 10a, it is possible to see the difference between the progression of the eigenvalue over indexes and the respective regularized eigenvalue.

Figure 9: Eigenvalue progression over iteration versus the regularized eigenvalue progression over iteration.

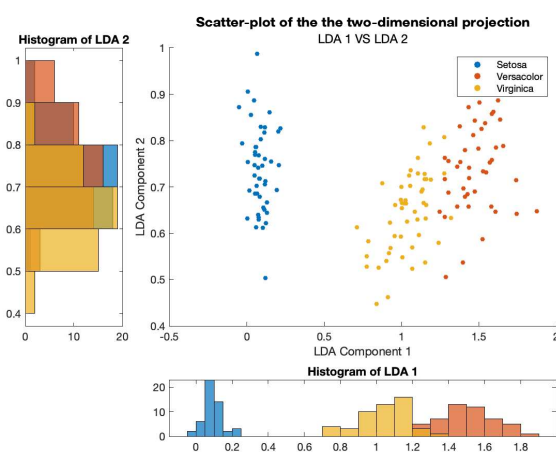


(a) Eigenvalue progression

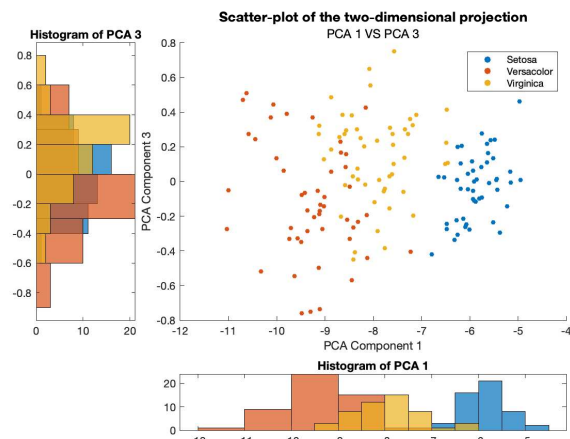
PCA vs LDA. Difference between PCA versus LDA method: while the **Principal Component Analysis** (PCA) is a method to look the data in a low dimensional subspace where the spread of the data in their principal direction is maximized; **Linear Discriminant Analysis** is a method for looking the data into a low dimensional subspace but into directions that minimizes the spread of each cluster and, simultaneously, maximizes the separation between each cluster. That's if data suggest cluster, PCA doesn't guarantee that the clusters are visible in the projection of the data onto the direction.



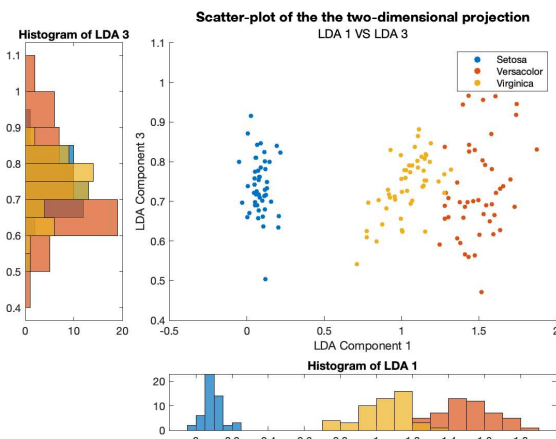
(b) PCA 1 vs PCA 2



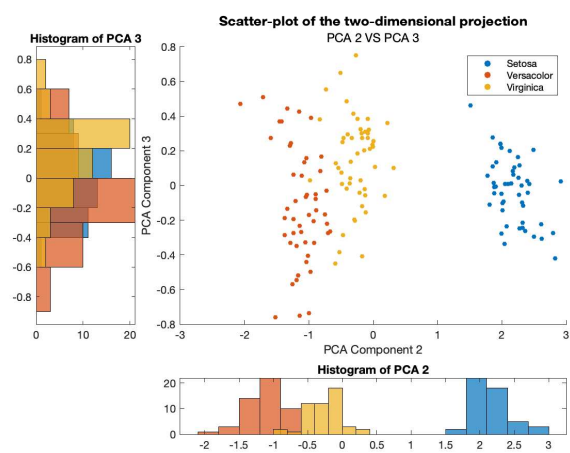
(c) LDA 1 vs LDA 2



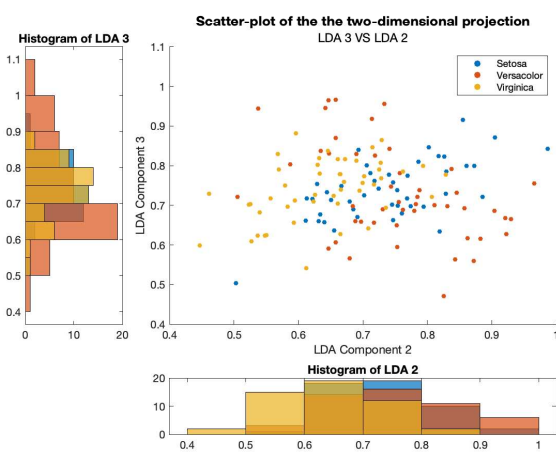
(d) PCA 1 vs PCA 3



(e) LDA 1 vs LDA 3



(f) PCA 2 vs PCA 3



(g) LDA 2 vs LDA 3

Appendices

Appendix

Wine Dataset.

PCA.

```

1 %% PCA - Wine Data
2 clear all
3 close all
4
5 load("WineData.mat")
6
7 I1 = find(I == 1);
8 I2 = find(I == 2);
9 I3 = find(I == 3);
10
11 %SVD
12 [U, D, ~] = svd(X, 'econ');
13 Z = U(:, 1:3)' * X;
14
15 % Histogram for PCA component 1
16 figure;
17 t = tiledlayout(4, 4, 'TileSpacing', 'Compact', 'Padding', 'Compact');
18
19 ax1 = nexttile(1, [3, 1]);
20 histogram(Z(1, I1), 'FaceColor', [0 0.4470 0.7410], 'FaceAlpha', 0.7)
21 hold on;
22 histogram(Z(1, I2), 'FaceColor', [0.8500 0.3250 0.0980], 'FaceAlpha',
23 0.7)
24 histogram(Z(1, I3), 'FaceColor', [0.9290 0.6940 0.1250], 'FaceAlpha',
25 0.7)
26 view(ax1, 90, -90); % Rotate the histogram to align it vertically
27 title('Histogram of PCA 1');
28 hold off;
29
30 % Scatter plot of PCA components 1 and 2
31 ax2 = nexttile(2, [3, 3]);
32 scatter(Z(1, I1), Z(2, I1), 20, 'filled', "MarkerFaceColor", [0 0.4470
33 0.7410])
34 hold on;
35 scatter(Z(1, I2), Z(2, I2), 20, 'filled', "MarkerFaceColor", [0.8500
36 0.3250 0.0980])
37 scatter(Z(1, I3), Z(2, I3), 20, 'filled', "MarkerFaceColor", [0.9290
38 0.6940 0.1250])
39 xlabel('PCA Component 1', 'FontSize', 11);
40 ylabel('PCA Component 2', 'FontSize', 11);
41 legend('Cultivar 1', 'Cultivar 2', 'Cultivar 3');
42 hold off;
43 title('Scatter-plot of the two-dimensional projection', 'FontSize', 13)
44 subtitle('PCA 1 VS PCA 2', 'FontSize', 11.5)
45
46 % Histogram for PCA component 2 (below x-axis)
47 ax3 = nexttile(14, [1, 3]);
48 histogram(Z(2, I1), 'FaceColor', [0 0.4470 0.7410], 'FaceAlpha', 0.7)
49 hold on;

```

```

46 histogram(Z(2, I2), 'FaceColor', [0.8500 0.3250 0.0980], 'FaceAlpha',
    0.7)
47 histogram(Z(2, I3), 'FaceColor', [0.9290 0.6940 0.1250], 'FaceAlpha',
    0.7)
48 title('Histogram of PCA 2');
49 hold off;
50
51 % Plot projections on PCA components 1 and 3
52 figure;
53 t = tiledlayout(4, 4, 'TileSpacing', 'Compact', 'Padding', 'Compact');
54
55 % Histogram for PCA component 3 (left y-axis)
56 ax1 = nexttile(1, [3, 1]);
57 histogram(Z(3, I1), 'FaceColor', [0 0.4470 0.7410], 'FaceAlpha', 0.7)
58 hold on;
59 histogram(Z(3, I2), 'FaceColor', [0.8500 0.3250 0.0980], 'FaceAlpha',
    0.7)
60 histogram(Z(3, I3), 'FaceColor', [0.9290 0.6940 0.1250], 'FaceAlpha',
    0.7)
61 view(ax1, 90, -90); % Rotate the histogram to align it vertically
62 title('Histogram of PCA 3');
63 hold off;
64
65 % Scatter plot of PCA components 1 and 3
66 ax2 = nexttile(2, [3, 3]);
67 scatter(Z(1, I1), Z(3, I1), 20, 'filled', "MarkerFaceColor", [0 0.4470
    0.7410])
68 hold on;
69 scatter(Z(1, I2), Z(3, I2), 20, 'filled', "MarkerFaceColor", [0.8500
    0.3250 0.0980])
70 scatter(Z(1, I3), Z(3, I3), 20, 'filled', "MarkerFaceColor", [0.9290
    0.6940 0.1250])
71 xlabel('PCA Component 1', 'FontSize', 11);
72 ylabel('PCA Component 3', 'FontSize', 11);
73 legend('Cultivar 1', 'Cultivar 2', 'Cultivar 3');
74 hold off;
75 title('Scatter-plot of the two-dimensional projection', 'FontSize', 13)
76 subtitle('PCA 1 VS PCA 3', 'FontSize', 11.5)
77
78 % Histogram for PCA component 1 (below x-axis)
79 ax3 = nexttile(14, [1, 3]);
80 histogram(Z(1, I1), 'FaceColor', [0 0.4470 0.7410], 'FaceAlpha', 0.7)
81 hold on;
82 histogram(Z(1, I2), 'FaceColor', [0.8500 0.3250 0.0980], 'FaceAlpha',
    0.7)
83 histogram(Z(1, I3), 'FaceColor', [0.9290 0.6940 0.1250], 'FaceAlpha',
    0.7)
84 title('Histogram of PCA 1');
85 hold off;
86
87 % Plot projections on PCA components 2 and 3
88 figure;
89 t = tiledlayout(4, 4, 'TileSpacing', 'Compact', 'Padding', 'Compact');
90
91 % Histogram for PCA component 3 (left y-axis)
92 ax1 = nexttile(1, [3, 1]);
93 histogram(Z(3, I1), 'FaceColor', [0 0.4470 0.7410], 'FaceAlpha', 0.7)
94 hold on;

```

```

95 histogram(Z(3, I2), 'FaceColor', [0.8500 0.3250 0.0980], 'FaceAlpha',
    0.7)
96 histogram(Z(3, I3), 'FaceColor', [0.9290 0.6940 0.1250], 'FaceAlpha',
    0.7)
97 view(ax1, 90, -90); % Rotate the histogram to align it vertically
98 title('Histogram of PCA 3');
99 hold off;
100
101 % Scatter plot of PCA components 2 and 3
102 ax2 = nexttile(2, [3, 3]);
103 scatter(Z(2, I1), Z(3, I1), 20, 'filled', "MarkerFaceColor", [0 0.4470
    0.7410])
104 hold on;
105 scatter(Z(2, I2), Z(3, I2), 20, 'filled', "MarkerFaceColor", [0.8500
    0.3250 0.0980])
106 scatter(Z(2, I3), Z(3, I3), 20, 'filled', "MarkerFaceColor", [0.9290
    0.6940 0.1250])
107 xlabel('PCA Component 2', 'FontSize', 11);
108 ylabel('PCA Component 3', 'FontSize', 11);
109 legend('Cultivar 1', 'Cultivar 2', 'Cultivar 3');
110 hold off;
111 title('Scatter-plot of the two-dimensional projection', 'FontSize', 13)
112 subtitle('PCA 2 VS PCA 3', 'FontSize', 11.5)
113
114 % Histogram for PCA component 2 (below x-axis)
115 ax3 = nexttile(14, [1, 3]);
116 histogram(Z(2, I1), 'FaceColor', [0 0.4470 0.7410], 'FaceAlpha', 0.7)
117 hold on;
118 histogram(Z(2, I2), 'FaceColor', [0.8500 0.3250 0.0980], 'FaceAlpha',
    0.7)
119 histogram(Z(2, I3), 'FaceColor', [0.9290 0.6940 0.1250], 'FaceAlpha',
    0.7)
120 title('Histogram of PCA 2');
121 hold off;

```

LDA.

```

1 %% LDA - Wine Dataset
2 clear all
3 close all
4
5
6 load("WineData.mat")
7
8 I1 = find(I == 1);
9 I2 = find(I == 2);
10 I3 = find(I == 3);
11
12 X1 = X(:, I1);
13 X2 = X(:, I2);
14 X3 = X(:, I3);
15
16 %cluster centroid
17 p1 = length(I1);
18 c1 = mean(X1, 2);
19 X1c = X1 - c1;
20

```

```

21 p2 = length(I2);
22 c2 = mean(X2, 2);
23 X2c = X2 - c2;
24
25 p3 = length(I3);
26 c3 = mean(X3, 2);
27 X3c = X3 - c3;
28
29 %the within-cluster scatter matrix
30 Sw = X1c * X1c' + X2c * X2c' + X3c * X3c';
31
32 %the between-cluster scatter matrix
33 p = p1 + p2 + p3;
34 c = mean(X, 2);
35 Sb = p1 * (c1 - c) * (c1 - c)' + p2 * (c2 - c) * (c2 - c)' + p3 * (c3 -
    c) * (c3 - c)';
36
37 % Check if regularization is needed
38 epsilon = 1e-10;
39 if det(Sw) <= 1e-14
40     disp('Regularization needed')
41     Sw = Sw + epsilon * eye(size(Sw));
42 end
43
44 % Cholesky factorization of the within-cluster scatter matrix
45 K = chol(Sw, 'lower');
46 A = (K' \ Sb) / K;
47
48 %eigenvalues and eigenvectors of the matrix A
49 [W, D] = eig(A);
50 [D_sorted, idx] = sort(diag(D), 'descend');
51 W_lda = W(:, idx);
52
53 % Project data onto new feature space
54 Q = inv(K) * W_lda;
55 LDA_projection = Q(:, 1:3)' * X;
56
57 % Plot the results
58 figure;
59 t = tiledlayout(4, 4, 'TileSpacing', 'Compact', 'Padding', 'Compact');
60
61 % Histogram for LDA component 2 (left y-axis)
62 ax1 = nexttile(1, [3, 1]);
63 histogram(LDA_projection(2, I1), 'FaceColor', [0 0.4470 0.7410], '
    FaceAlpha', 0.7)
64 hold on;
65 histogram(LDA_projection(2, I2), 'FaceColor', [0.8500 0.3250 0.0980], '
    FaceAlpha', 0.7)
66 histogram(LDA_projection(2, I3), 'FaceColor', [0.9290 0.6940 0.1250], '
    FaceAlpha', 0.7)
67 view(ax1, 90, -90)
68 title('Histogram of LDA 2');
69 hold off;
70
71 % Scatter plot of LDA components 1 and 2
72 ax2 = nexttile(2, [3, 3]);
73 scatter(LDA_projection(1, I1), LDA_projection(2, I1), 20, 'filled', "
    MarkerFaceColor", [0 0.4470 0.7410])

```



```

74 hold on;
75 scatter(LDA_projection(1, I2), LDA_projection(2, I2), 20, 'filled', "
    MarkerFaceColor", [0.8500 0.3250 0.0980])
76 scatter(LDA_projection(1, I3), LDA_projection(2, I3), 20, 'filled', "
    MarkerFaceColor", [0.9290 0.6940 0.1250])
77 xlabel('LDA Component 1', 'FontSize', 11);
78 ylabel('LDA Component 2', 'FontSize', 11);
79 legend('Cultivar 1', 'Cultivar 2', 'Cultivar 3');
80 hold off;
81 title('Scatter-plot of the the two-dimensional projection', 'FontSize',
    13)
82 subtitle('LDA 1 VS LDA 2', 'FontSize', 11.5)
83
84 % Histogram for LDA component 1 (below x-axis)
85 ax3 = nexttile(14, [1, 3]);
86 histogram(LDA_projection(1, I1), 'FaceColor', [0 0.4470 0.7410], '
    FaceAlpha', 0.7)
87 hold on;
88 histogram(LDA_projection(1, I2), 'FaceColor', [0.8500 0.3250 0.0980], '
    FaceAlpha', 0.7)
89 histogram(LDA_projection(1, I3), 'FaceColor', [0.9290 0.6940 0.1250], '
    FaceAlpha', 0.7)
90 title('Histogram of LDA 1');
91 hold off;
92
93 % Plot projections on LDA components 1 and 3
94 figure;
95 t = tiledlayout(4, 4, 'TileSpacing', 'Compact', 'Padding', 'Compact');
96
97 % Histogram for LDA component 3 (left y-axis)
98 ax1 = nexttile(1, [3, 1]);
99 histogram(LDA_projection(3, I1), 'FaceColor', [0 0.4470 0.7410], '
    FaceAlpha', 0.7)
100 hold on;
101 histogram(LDA_projection(3, I2), 'FaceColor', [0.8500 0.3250 0.0980], '
    FaceAlpha', 0.7)
102 histogram(LDA_projection(3, I3), 'FaceColor', [0.9290 0.6940 0.1250], '
    FaceAlpha', 0.7)
103 view(ax1, 90, -90)
104 title('Histogram of LDA 3');
105 hold off;
106
107 % Scatter plot of LDA components 1 and 3
108 ax2 = nexttile(2, [3, 3]);
109 scatter(LDA_projection(1, I1), LDA_projection(3, I1), 20, 'filled', "
    MarkerFaceColor", [0 0.4470 0.7410])
110 hold on;
111 scatter(LDA_projection(1, I2), LDA_projection(3, I2), 20, 'filled', "
    MarkerFaceColor", [0.8500 0.3250 0.0980])
112 scatter(LDA_projection(1, I3), LDA_projection(3, I3), 20, 'filled', "
    MarkerFaceColor", [0.9290 0.6940 0.1250])
113 xlabel('LDA Component 1', 'FontSize', 11);
114 ylabel('LDA Component 3', 'FontSize', 11);
115 legend('Cultivar 1', 'Cultivar 2', 'Cultivar 3');
116 hold off;
117 title('Scatter-plot of the the two-dimensional projection', 'FontSize',
    13)
118 subtitle('LDA 3 VS LDA 1', 'FontSize', 11.5)

```

```

119
120 % Histogram for LDA component 1 (below x-axis)
121 ax3 = nexttile(14, [1, 3]);
122 histogram(LDA_projection(1, I1), 'FaceColor', [0 0.4470 0.7410], '
    FaceAlpha', 0.7)
123 hold on;
124 histogram(LDA_projection(1, I2), 'FaceColor', [0.8500 0.3250 0.0980], '
    FaceAlpha', 0.7)
125 histogram(LDA_projection(1, I3), 'FaceColor', [0.9290 0.6940 0.1250], '
    FaceAlpha', 0.7)
126 title('Histogram of LDA 1');
127 hold off;
128
129 % Plot projections on LDA components 2 and 3
130 figure;
131 t = tiledlayout(4, 4, 'TileSpacing', 'Compact', 'Padding', 'Compact');
132
133 % Histogram for LDA component 3 (left y-axis)
134 ax1 = nexttile(1, [3, 1]);
135 histogram(LDA_projection(3, I1), 'FaceColor', [0 0.4470 0.7410], '
    FaceAlpha', 0.7)
136 hold on;
137 histogram(LDA_projection(3, I2), 'FaceColor', [0.8500 0.3250 0.0980], '
    FaceAlpha', 0.7)
138 histogram(LDA_projection(3, I3), 'FaceColor', [0.9290 0.6940 0.1250], '
    FaceAlpha', 0.7)
139 view(ax1, 90, -90)
140 title('Histogram of LDA 3');
141 hold off;
142
143 % Scatter plot of LDA components 2 and 3
144 ax2 = nexttile(2, [3, 3]);
145 scatter(LDA_projection(2, I1), LDA_projection(3, I1), 20, 'filled', "
    MarkerFaceColor", [0 0.4470 0.7410])
146 hold on;
147 scatter(LDA_projection(2, I2), LDA_projection(3, I2), 20, 'filled', "
    MarkerFaceColor", [0.8500 0.3250 0.0980])
148 scatter(LDA_projection(2, I3), LDA_projection(3, I3), 20, 'filled', "
    MarkerFaceColor", [0.9290 0.6940 0.1250])
149 xlabel('LDA Component 2', 'FontSize', 11);
150 ylabel('LDA Component 3', 'FontSize', 11);
151 legend('Cultivar 1', 'Cultivar 2', 'Cultivar 3');
152 hold off;
153 title('Scatter-plot of the the two-dimensional projection', 'FontSize',
    13)
154 subtitle('LDA 3 VS LDA 2', 'FontSize', 11.5)
155
156 % Histogram for LDA component 2 (below x-axis)
157 ax3 = nexttile(14, [1, 3]);
158 histogram(LDA_projection(2, I1), 'FaceColor', [0 0.4470 0.7410], '
    FaceAlpha', 0.7)
159 hold on;
160 histogram(LDA_projection(2, I2), 'FaceColor', [0.8500 0.3250 0.0980], '
    FaceAlpha', 0.7)
161 histogram(LDA_projection(2, I3), 'FaceColor', [0.9290 0.6940 0.1250], '
    FaceAlpha', 0.7)
162 title('Histogram of LDA 2');
163 hold off;

```

```

164
165 % Plotting the eigenvalues with and without regularization
166 figure;
167 eig_Sw = eig(Sw);
168 semilogy(eig_Sw, '*');
169 hold on;
170 semilogy(eig_Sw + epsilon, 'sm');
171 legend('eigenvalues', 'regularized eigenvalues');
172 title('Eigenvalue of Sw vs regularized eigenvalue', 'FontSize', 13);
173 xlabel('Index', 'FontSize', 11); % Adding x-axis label
174 ylabel('Eigenvalue', 'FontSize', 11); % Adding y-axis label
175 hold off;
176
177 % Check determinant before and after regularization
178 disp('det(Sw) before regularization');
179 det_Sw = det(Sw);
180 disp(det_Sw);
181
182 if (det_Sw <= 1e-14)
183     disp('Regularization applied');
184     Sw = Sw + epsilon * eye(size(Sw));
185     disp('det(Sw) after regularization');
186     det(Sw)
187 end
188
189 % Plotting eigenvalues before and after regularization
190 figure;
191 sSw = eig(Sw);
192 sSw_reg = eig(Sw + epsilon * eye(size(Sw)));
193 semilogy(sSw, 'r*');
194 hold on;
195 semilogy(sSw_reg, 'bd');
196 legend('eig before', 'eig after');
197 title('Eigenvalue of Sw before and after regularization', 'FontSize',
198     13);
199 xlabel('Index', 'FontSize', 11); % Adding x-axis label
200 ylabel('Eigenvalue', 'FontSize', 11); % Adding y-axis label
201 hold off;

```

HandWrittenDigit Dataset.

PCA.

```

1 %% PCA - Handwritten Digits
2 clear all
3 close all
4
5
6 load('HandwrittenDigits.mat')
7
8
9 I0 = find(I == 0);
10 I4 = find(I == 4);
11
12 %SVD
13 [U, D, ~] = svd(X, 'econ');

```

```

14 Z = U(:, 1:3)' * X;
15
16
17 % Histogram for PCA component 1
18 figure;
19 t = tiledlayout(4, 4, 'TileSpacing', 'Compact', 'Padding', 'Compact');
20
21 ax1 = nexttile(1, [3, 1]);
22 histogram(Z(1, I0), 'FaceColor', [0 0.4470 0.7410], 'FaceAlpha', 0.7)
23 hold on;
24 histogram(Z(1, I4), 'FaceColor', [0.8500 0.3250 0.0980], 'FaceAlpha',
    0.7)
25 view(ax1, 90, -90); % Rotate the histogram to align it vertically
26 title('Histogram of PCA 1');
27 hold off;
28
29 % Scatter plot of PCA components 1 and 2
30 ax2 = nexttile(2, [3, 3]);
31 scatter(Z(1, I0), Z(2, I0), 20, 'filled', "MarkerFaceColor", [0 0.4470
    0.7410])
32 hold on;
33 scatter(Z(1, I4), Z(2, I4), 20, 'filled', "MarkerFaceColor", [0.8500
    0.3250 0.0980])
34 xlabel('PCA Component 1', 'FontSize', 11);
35 ylabel('PCA Component 2', 'FontSize', 11);
36 legend('Digit 0', 'Digit 4');
37 hold off;
38 title('Scatter-plot of PCA 1 VS PCA 2', 'FontSize', 13)
39
40 % Histogram for PCA component 2 (below x-axis)
41 ax3 = nexttile(14, [1, 3]);
42 histogram(Z(2, I0), 'FaceColor', [0 0.4470 0.7410], 'FaceAlpha', 0.7)
43 hold on;
44 histogram(Z(2, I4), 'FaceColor', [0.8500 0.3250 0.0980], 'FaceAlpha',
    0.7)
45 title('Histogram of PCA 2');
46 hold off;
47
48 % Plot projections on PCA components 1 and 3
49 figure;
50 t = tiledlayout(4, 4, 'TileSpacing', 'Compact', 'Padding', 'Compact');
51
52 % Histogram for PCA component 3 (left y-axis)
53 ax1 = nexttile(1, [3, 1]);
54 histogram(Z(3, I0), 'FaceColor', [0 0.4470 0.7410], 'FaceAlpha', 0.7)
55 hold on;
56 histogram(Z(3, I4), 'FaceColor', [0.8500 0.3250 0.0980], 'FaceAlpha',
    0.7)
57 view(ax1, 90, -90); % Rotate the histogram to align it vertically
58 title('Histogram of PCA 3');
59 hold off;
60
61 % Scatter plot of PCA components 1 and 3
62 ax2 = nexttile(2, [3, 3]);
63 scatter(Z(1, I0), Z(3, I0), 20, 'filled', "MarkerFaceColor", [0 0.4470
    0.7410])
64 hold on;

```

```

65 scatter(Z(1, I4), Z(3, I4), 20, 'filled', "MarkerFaceColor", [0.8500
    0.3250 0.0980])
66 xlabel('PCA Component 1', 'FontSize', 11);
67 ylabel('PCA Component 3', 'FontSize', 11);
68 legend('Digit 0', 'Digit 4');
69 hold off;
70 title('Scatter-plot of PCA 1 VS PCA 3', 'FontSize', 13)
71
72 % Histogram for PCA component 1 (below x-axis)
73 ax3 = nexttile(14, [1, 3]);
74 histogram(Z(1, I0), 'FaceColor', [0 0.4470 0.7410], 'FaceAlpha', 0.7)
75 hold on;
76 histogram(Z(1, I4), 'FaceColor', [0.8500 0.3250 0.0980], 'FaceAlpha',
    0.7)
77 title('Histogram of PCA 1');
78 hold off;
79
80 % Plot projections on PCA components 2 and 3
81 figure;
82 t = tiledlayout(4, 4, 'TileSpacing', 'Compact', 'Padding', 'Compact');
83
84 % Histogram for PCA component 3 (left y-axis)
85 ax1 = nexttile(1, [3, 1]);
86 histogram(Z(3, I0), 'FaceColor', [0 0.4470 0.7410], 'FaceAlpha', 0.7)
87 hold on;
88 histogram(Z(3, I4), 'FaceColor', [0.8500 0.3250 0.0980], 'FaceAlpha',
    0.7)
89 view(ax1, 90, -90); % Rotate the histogram to align it vertically
90 title('Histogram of PCA 3');
91 hold off;
92
93 % Scatter plot of PCA components 2 and 3
94 ax2 = nexttile(2, [3, 3]);
95 scatter(Z(2, I0), Z(3, I0), 20, 'filled', "MarkerFaceColor", [0 0.4470
    0.7410])
96 hold on;
97 scatter(Z(2, I4), Z(3, I4), 20, 'filled', "MarkerFaceColor", [0.8500
    0.3250 0.0980])
98 xlabel('PCA Component 2', 'FontSize', 11);
99 ylabel('PCA Component 3', 'FontSize', 11);
100 legend('Digit 0', 'Digit 4');
101 hold off;
102 title('Scatter-plot of PCA 2 VS PCA 3', 'FontSize', 13)
103
104 % Histogram for PCA component 2 (below x-axis)
105 ax3 = nexttile(14, [1, 3]);
106 histogram(Z(2, I0), 'FaceColor', [0 0.4470 0.7410], 'FaceAlpha', 0.7)
107 hold on;
108 histogram(Z(2, I4), 'FaceColor', [0.8500 0.3250 0.0980], 'FaceAlpha',
    0.7)
109 title('Histogram of PCA 2');
110 hold off;

```

LDA.

```

1 %% LDA - HandWrittenDigits
2

```

```

3 clear all
4 close all
5
6 load('HandwrittenDigits.mat')
7
8 I0 = find(I == 0);
9 I4 = find(I == 4);
10
11 X0 = X(:, I0);
12 X4 = X(:, I4);
13 X04 = [X0, X4];
14
15 %Cluster centroid
16 p1 = length(I0);
17 c1 = mean(X0, 2);
18 X1c = X0 - c1;
19
20 p2 = length(I4);
21 c2 = mean(X4, 2);
22 X2c = X4 - c2;
23
24 %the within-class scatter matrix
25 Sw = X1c * X1c' + X2c * X2c';
26
27 %the between-class scatter matrix
28 p = p1 + p2;
29 c = mean(X04, 2);
30 Sb = p1 * (c1 - c) * (c1 - c)' + p2 * (c2 - c) * (c2 - c)';
31
32 % Check if regularization is needed
33 epsilon = 1e-10;
34 if det(Sw) <= 1e-14
35     disp('Regularization needed')
36     Sw = Sw + epsilon * eye(size(Sw));
37 end
38
39 % Cholesky factorization of the within-cluster scatter matrix
40 K = chol(Sw, 'lower');
41 A = (K' \ Sb) / K;
42
43 %eigenvalues and eigenvectors of the matrix A
44 [W, D] = eig(A);
45 [D_sorted, idx] = sort(diag(D), 'descend');
46 W_lda = W(:, idx);
47
48 % Project data onto new feature space
49 Q = inv(K) * W_lda;
50 LDA_projection = Q(:, 1:3)' * X;
51
52 %Visualisation
53 figure;
54 t = tiledlayout(2, 3, 'TileSpacing', 'Compact', 'Padding', 'Compact');
55
56 % Scatter plot for LDA component 1
57 ax1 = nexttile(1, [1, 3]);
58 scatter(LDA_projection(1, I0), 20, 'filled', 'MarkerFaceColor', [0
    0.4470 0.7410]);
59 hold on;

```

```

60 scatter(LDA_projection(1, I4), 20, 'filled', 'MarkerFaceColor', [0.8500
    0.3250 0.0980]);
61 xlabel('LDA Component 1');
62 leg1 = scatter(NaN, NaN, 'filled', 'MarkerFaceColor', [0 0.4470
    0.7410], 'DisplayName', 'Digit 0');
63 leg2 = scatter(NaN, NaN, 'filled', 'MarkerFaceColor', [0.8500 0.3250
    0.0980], 'DisplayName', 'Digit 4');
64 legend([leg1, leg2], 'Location', 'best');
65 legend('boxoff');
66 hold off;
67
68 % Histogram for LDA component 1
69 ax2 = nexttile(4, [1, 3]);
70 real_LDA_projection_1_I0 = real(LDA_projection(1, I0));
71 real_LDA_projection_1_I4 = real(LDA_projection(1, I4));
72 histogram(real_LDA_projection_1_I0, 'FaceColor', [0 0.4470 0.7410], '
    FaceAlpha', 0.7);
73 hold on;
74 histogram(real_LDA_projection_1_I4, 'FaceColor', [0.8500 0.3250
    0.0980], 'FaceAlpha', 0.7);
75 hold off;
76
77 % Plot the projections on the second LDA direction
78 figure;
79 t = tiledlayout(2, 3, 'TileSpacing', 'Compact', 'Padding', 'Compact');
80
81 % Scatter plot for LDA component 2
82 ax1 = nexttile(1, [1, 3]);
83 scatter(LDA_projection(2, I0), 20, 'filled', 'MarkerFaceColor', [0
    0.4470 0.7410]);
84 hold on;
85 scatter(LDA_projection(2, I4), 20, 'filled', 'MarkerFaceColor', [0.8500
    0.3250 0.0980]);
86 xlabel('LDA Component 2');
87 leg1 = scatter(NaN, NaN, 'filled', 'MarkerFaceColor', [0 0.4470
    0.7410], 'DisplayName', 'Digit 0');
88 leg2 = scatter(NaN, NaN, 'filled', 'MarkerFaceColor', [0.8500 0.3250
    0.0980], 'DisplayName', 'Digit 4');
89 legend([leg1, leg2], 'Location', 'best');
90 legend('boxoff');
91 hold off;
92
93 % Histogram for LDA component 2
94 ax2 = nexttile(4, [1, 3]);
95 real_LDA_projection_2_I0 = real(LDA_projection(2, I0));
96 real_LDA_projection_2_I4 = real(LDA_projection(2, I4));
97 histogram(real_LDA_projection_2_I0, 'FaceColor', [0 0.4470 0.7410], '
    FaceAlpha', 0.7);
98 hold on;
99 histogram(real_LDA_projection_2_I4, 'FaceColor', [0.8500 0.3250
    0.0980], 'FaceAlpha', 0.7);
100 hold off;
101
102 % Plotting the eigenvalues with and without regularization
103 figure;
104 eig_Sw = eig(Sw);
105 semilogy(eig_Sw, '*');
106 hold on;

```

```

107 semilogy(eig_Sw + epsilon, 'sm');
108 legend('eigenvalues', 'regularized eigenvalues');
109 title('Eigenvalue of Sw vs regularized eigenvalue', 'FontSize', 13);
110 xlabel('Index', 'FontSize', 11); % Adding x-axis label
111 ylabel('Eigenvalue', 'FontSize', 11); % Adding y-axis label
112 hold off;
113
114 % Check determinant before and after regularization
115 disp('det(Sw) before regularization');
116 det_Sw = det(Sw);
117 disp(det_Sw);
118
119 if (det_Sw <= 1e-14)
120     disp('Regularization applied');
121     Sw = Sw + epsilon * eye(size(Sw));
122     disp('det(Sw) after regularization');
123     det(Sw)
124 end
125
126 % Plotting eigenvalues before and after regularization
127 figure;
128 sSw = eig(Sw);
129 sSw_reg = eig(Sw + epsilon * eye(size(Sw)));
130 semilogy(sSw, 'r*');
131 hold on;
132 semilogy(sSw_reg, 'bd');
133 legend('eig before', 'eig after');
134 title('Eigenvalue of Sw before and after regularization', 'FontSize',
135       13);
136 xlabel('Index', 'FontSize', 11); % Adding x-axis label
137 ylabel('Eigenvalue', 'FontSize', 11); % Adding y-axis label
138 hold off;

```

NMF.

```

1 %% NMF - ANLS - HandWrittenDigits
2 close all
3 clear all
4 load HandwrittenDigits.mat
5
6
7 I0 = find(I == 0);
8 I4 = find(I == 4);
9
10 X0 = X(:,I0);
11 X4 = X(:,I4);
12 X04 = [X0, X4];
13
14 % Non-negative Matrix Factorization (NMF)
15 [n, p] = size(X04);
16 k_values = [5, 10, 20];
17
18 % Loop over different values of k
19 for k = k_values
20     disp(['k = ', num2str(k)]);
21     tau = 0.01; % Stopping criterion: iterate until relative change
                below this tolerance

```



```

22     ch_curr = 1;
23
24     % Initialize the matrices with random values
25     rng(1); % Seed for reproducibility
26     W = rand(n, k);
27     H = rand(k, p);
28
29     % Scale the feature vectors so that columns of W have unit l1 norm
30     Wnorms = max(W);
31     W = W * diag(1 ./ Wnorms);
32
33     nmax = 100; % Maximum number of iterations
34     t = 0;
35     ch = NaN(1, nmax); % Array to store changes for plotting
36     convergence
37
38     % Iterate until the relative change is below tau or the maximum
39     % number of iterations
40     while ch_curr > tau && t < nmax
41         % Update H
42         H_old = H;
43         for j = 1:p
44             H(:, j) = lsqnonneg(W, X04(:, j));
45         end
46         dH = norm(H - H_old, 'fro') / norm(H_old, 'fro'); % Relative
47         change in H
48
49         % Update W
50         W_old = W;
51         for j = 1:n
52             Xr = X04(j, :);
53             Wrt = lsqnonneg(H', Xr');
54             W(j, :) = Wrt';
55         end
56         dW = norm(W - W_old, 'fro') / norm(W_old, 'fro'); % Relative
57         change in W
58
59         % Update the current change and iteration counter
60         ch_curr = dH + dW;
61         t = t + 1;
62         ch(t) = ch_curr;
63     end
64
65     % Plot the convergence diagnostics (relative change) during the
66     % iterations
67     figure(k);
68     semilogy(ch(2:t), 'k-', 'LineWidth', 2);
69     hold on;
70     semilogy(ch(2:t), 'r.', 'MarkerSize', 12);
71     hold off;
72     title('Convergence diagnostics', 'FontSize', 13);
73     xlabel('Iteration', 'FontSize', 11);
74     ylabel('Relative change', 'FontSize', 11);
75
76     % Number of iterations
77     disp(['Iterations = ', num2str(t - 1)]);
78
79     % Residual error

```

```

75     res = X04 - W * H;
76     rel_res = norm(res, 'fro') / norm(X04, 'fro');
77     disp(['Relative residual = ', num2str(100 * rel_res, 3), '%']);
78
79     % Plot the feature vectors as images
80     figure(k + 1);
81     tiledlayout('flow');
82     for j = 1:k
83         nexttile;
84         imagesc(reshape(W(:, j), 16, 16));
85         axis('square');
86         axis('off');
87         colormap(1 - gray);
88     end
89     sgtitle(['Basis Vectors for k = ', num2str(k)], 'FontSize', 13);
90 end
91
92 % Identify the first feature vector
93 selected_feature_vector = 1;
94 coefficients = H(selected_feature_vector, :);
95 num_zeros = 0;
96 num_fours = 0;
97
98 % The first 319 elements in the "coefficients" correspond to '0', the
99 % others to '4'
100 for i = 1:length(coefficients)
101     if i <= length(I0)
102         % Corresponding to a '0'
103         if coefficients(i) > 0.1 % lower than 0.1 is not significant
104             num_zeros = num_zeros + 1;
105         end
106     else
107         % Corresponding to a '4'
108         if coefficients(i) > 0.1
109             num_fours = num_fours + 1;
110         end
111     end
112 end
113 disp(['Number of significant coefficients for "0": ', num2str(num_zeros)
114     ]);
115 disp(['Number of significant coefficients for "4": ', num2str(num_fours)
116     ]);
117 if num_zeros > num_fours
118     disp('The selected feature vector represents a "0".');
119 else
120     disp('The selected feature vector represents a "4".');
121 end

```

Iris Dataset.

PCA.

```

1 %% PCA - Iris
2
3 clear all

```

```

4 close all
5
6 load IrisDataAnnotated.mat
7
8 I1 = find(I == 1);
9 I2 = find(I == 2);
10 I3 = find(I == 3);
11
12 %SVD
13 [U, D, ~] = svd(X, 'econ');
14 Z = U(:, 1:3)' * X;
15
16 % Histogram for PCA component 1
17 figure;
18 t = tiledlayout(4, 4, 'TileSpacing', 'Compact', 'Padding', 'Compact');
19
20 ax1 = nexttile(1, [3, 1]);
21 histogram(Z(1, I1), 'FaceColor', [0 0.4470 0.7410], 'FaceAlpha', 0.7)
22 hold on;
23 histogram(Z(1, I2), 'FaceColor', [0.8500 0.3250 0.0980], 'FaceAlpha',
24           0.7)
25 histogram(Z(1, I3), 'FaceColor', [0.9290 0.6940 0.1250], 'FaceAlpha',
26           0.7)
27 view(ax1, 90, -90); % Rotate the histogram to align it vertically
28 title('Histogram of PCA 1');
29 hold off;
30
31 % Scatter plot of PCA components 1 and 2
32 ax2 = nexttile(2, [3, 3]);
33 scatter(Z(1, I1), Z(2, I1), 20, 'filled', "MarkerFaceColor", [0 0.4470
34           0.7410])
35 hold on;
36 scatter(Z(1, I2), Z(2, I2), 20, 'filled', "MarkerFaceColor", [0.8500
37           0.3250 0.0980])
38 scatter(Z(1, I3), Z(2, I3), 20, 'filled', "MarkerFaceColor", [0.9290
39           0.6940 0.1250])
40 xlabel('PCA Component 1', 'FontSize', 11);
41 ylabel('PCA Component 2', 'FontSize', 11);
42 legend('Setosa', 'Versicolor', 'Virginica');
43 hold off;
44 title('Scatter-plot of the two-dimensional projection', 'FontSize', 13)
45 subtitle('PCA 1 VS PCA 2', 'FontSize', 11.5)
46
47 % Histogram for PCA component 2 (below x-axis)
48 ax3 = nexttile(14, [1, 3]);
49 histogram(Z(2, I1), 'FaceColor', [0 0.4470 0.7410], 'FaceAlpha', 0.7)
50 hold on;
51 histogram(Z(2, I2), 'FaceColor', [0.8500 0.3250 0.0980], 'FaceAlpha',
52           0.7)
53 histogram(Z(2, I3), 'FaceColor', [0.9290 0.6940 0.1250], 'FaceAlpha',
54           0.7)
55 title('Histogram of PCA 2');
56 hold off;
57
58 % Plot projections on PCA components 1 and 3
59 figure;
60 t = tiledlayout(4, 4, 'TileSpacing', 'Compact', 'Padding', 'Compact');

```

```

55 % Histogram for PCA component 3 (left y-axis)
56 ax1 = nexttile(1, [3, 1]);
57 histogram(Z(3, I1), 'FaceColor', [0 0.4470 0.7410], 'FaceAlpha', 0.7)
58 hold on;
59 histogram(Z(3, I2), 'FaceColor', [0.8500 0.3250 0.0980], 'FaceAlpha',
60          0.7)
61 histogram(Z(3, I3), 'FaceColor', [0.9290 0.6940 0.1250], 'FaceAlpha',
62          0.7)
63 view(ax1, 90, -90); % Rotate the histogram to align it vertically
64 title('Histogram of PCA 3');
65 hold off;
66
67 % Scatter plot of PCA components 1 and 3
68 ax2 = nexttile(2, [3, 3]);
69 scatter(Z(1, I1), Z(3, I1), 20, 'filled', "MarkerFaceColor", [0 0.4470
70          0.7410])
71 hold on;
72 scatter(Z(1, I2), Z(3, I2), 20, 'filled', "MarkerFaceColor", [0.8500
73          0.3250 0.0980])
74 scatter(Z(1, I3), Z(3, I3), 20, 'filled', "MarkerFaceColor", [0.9290
75          0.6940 0.1250])
76 xlabel('PCA Component 1', 'FontSize', 11);
77 ylabel('PCA Component 3', 'FontSize', 11);
78 legend('Setosa', 'Versicolor', 'Virginica');
79 hold off;
80 title('Scatter-plot of the two-dimensional projection', 'FontSize', 13)
81 subtitle('PCA 1 VS PCA 3', 'FontSize', 11.5)
82
83 % Histogram for PCA component 1 (below x-axis)
84 ax3 = nexttile(14, [1, 3]);
85 histogram(Z(1, I1), 'FaceColor', [0 0.4470 0.7410], 'FaceAlpha', 0.7)
86 hold on;
87 histogram(Z(1, I2), 'FaceColor', [0.8500 0.3250 0.0980], 'FaceAlpha',
88          0.7)
89 histogram(Z(1, I3), 'FaceColor', [0.9290 0.6940 0.1250], 'FaceAlpha',
90          0.7)
91 title('Histogram of PCA 1');
92 hold off;
93
94 % Plot projections on PCA components 2 and 3
95 figure;
96 t = tiledlayout(4, 4, 'TileSpacing', 'Compact', 'Padding', 'Compact');
97
98 % Histogram for PCA component 3 (left y-axis)
99 ax1 = nexttile(1, [3, 1]);
100 histogram(Z(3, I1), 'FaceColor', [0 0.4470 0.7410], 'FaceAlpha', 0.7)
101 hold on;
102 histogram(Z(3, I2), 'FaceColor', [0.8500 0.3250 0.0980], 'FaceAlpha',
103          0.7)
104 histogram(Z(3, I3), 'FaceColor', [0.9290 0.6940 0.1250], 'FaceAlpha',
105          0.7)
106 view(ax1, 90, -90); % Rotate the histogram to align it vertically
107 title('Histogram of PCA 3');
108 hold off;
109
110 % Scatter plot of PCA components 2 and 3
111 ax2 = nexttile(2, [3, 3]);

```

```

103 scatter(Z(2, I1), Z(3, I1), 20, 'filled', "MarkerFaceColor", [0 0.4470
    0.7410])
104 hold on;
105 scatter(Z(2, I2), Z(3, I2), 20, 'filled', "MarkerFaceColor", [0.8500
    0.3250 0.0980])
106 scatter(Z(2, I3), Z(3, I3), 20, 'filled', "MarkerFaceColor", [0.9290
    0.6940 0.1250])
107 xlabel('PCA Component 2', 'FontSize', 11);
108 ylabel('PCA Component 3', 'FontSize', 11);
109 legend('Setosa', 'Versicolor', 'Virginica');
110 hold off;
111 title('Scatter-plot of the two-dimensional projection', 'FontSize', 13)
112 subtitle('PCA 2 VS PCA 3', 'FontSize', 11.5)
113
114 % Histogram for PCA component 2 (below x-axis)
115 ax3 = nexttile(14, [1, 3]);
116 histogram(Z(2, I1), 'FaceColor', [0 0.4470 0.7410], 'FaceAlpha', 0.7)
117 hold on;
118 histogram(Z(2, I2), 'FaceColor', [0.8500 0.3250 0.0980], 'FaceAlpha',
    0.7)
119 histogram(Z(2, I3), 'FaceColor', [0.9290 0.6940 0.1250], 'FaceAlpha',
    0.7)
120 title('Histogram of PCA 2');
121 hold off;

```

LDA.

```

1 %% LDA - Iris
2 clear all
3 close all
4
5
6 load IrisDataAnnotated.mat
7
8 I1 = find(I == 1);
9 I2 = find(I == 2);
10 I3 = find(I == 3);
11
12 X1 = X(:, I1);
13 X2 = X(:, I2);
14 X3 = X(:, I3);
15
16 %cluster centroid
17 p1 = length(I1);
18 c1 = mean(X1, 2);
19 X1c = X1 - c1;
20
21 p2 = length(I2);
22 c2 = mean(X2, 2);
23 X2c = X2 - c2;
24
25 p3 = length(I3);
26 c3 = mean(X3, 2);
27 X3c = X3 - c3;
28
29 %the within-cluster scatter matrix
30 Sw = X1c * X1c' + X2c * X2c' + X3c * X3c';

```

```

31
32 %the between-cluster scatter matrix
33 p = p1 + p2 + p3;
34 c = mean(X, 2);
35 Sb = p1 * (c1 - c) * (c1 - c)' + p2 * (c2 - c) * (c2 - c)' + p3 * (c3 -
    c) * (c3 - c)';
36
37 % Check if regularization is needed
38 epsilon = 1e-10;
39 if det(Sw) <= 1e-14
40     disp('Regularization needed')
41     Sw = Sw + epsilon * eye(size(Sw));
42 end
43
44 % Cholesky factorization of the within-cluster scatter matrix
45 K = chol(Sw, 'lower');
46 A = (K' \ Sb) / K;
47
48 %eigenvalues and eigenvectors of the matrix A
49 [W, D] = eig(A);
50 [D_sorted, idx] = sort(diag(D), 'descend');
51 W_lda = W(:, idx);
52
53 % Project data onto new feature space
54 Q = inv(K) * W_lda;
55 LDA_projection = Q(:, 1:3)' * X;
56
57 % Plot the results
58 figure;
59 t = tiledlayout(4, 4, 'TileSpacing', 'Compact', 'Padding', 'Compact');
60
61 % Histogram for LDA component 2 (left y-axis)
62 ax1 = nexttile(1, [3, 1]);
63 histogram(LDA_projection(2, I1), 'FaceColor', [0 0.4470 0.7410], '
    FaceAlpha', 0.7)
64 hold on;
65 histogram(LDA_projection(2, I2), 'FaceColor', [0.8500 0.3250 0.0980], '
    FaceAlpha', 0.7)
66 histogram(LDA_projection(2, I3), 'FaceColor', [0.9290 0.6940 0.1250], '
    FaceAlpha', 0.7)
67 view(ax1, 90, -90)
68 title('Histogram of LDA 2');
69 hold off;
70
71 % Scatter plot of LDA components 1 and 2
72 ax2 = nexttile(2, [3, 3]);
73 scatter(LDA_projection(1, I1), LDA_projection(2, I1), 20, 'filled', "
    MarkerFaceColor", [0 0.4470 0.7410])
74 hold on;
75 scatter(LDA_projection(1, I2), LDA_projection(2, I2), 20, 'filled', "
    MarkerFaceColor", [0.8500 0.3250 0.0980])
76 scatter(LDA_projection(1, I3), LDA_projection(2, I3), 20, 'filled', "
    MarkerFaceColor", [0.9290 0.6940 0.1250])
77 xlabel('LDA Component 1', 'FontSize', 11);
78 ylabel('LDA Component 2', 'FontSize', 11);
79 legend('Setosa', 'Versicolor', 'Virginica');
80 hold off;

```

```

81 title('Scatter-plot of the the two-dimensional projection', 'FontSize',
    13)
82 subtitle('LDA 1 VS LDA 2', 'FontSize', 11.5)
83
84 % Histogram for LDA component 1 (below x-axis)
85 ax3 = nexttile(14, [1, 3]);
86 histogram(LDA_projection(1, I1), 'FaceColor', [0 0.4470 0.7410], '
    FaceAlpha', 0.7)
87 hold on;
88 histogram(LDA_projection(1, I2), 'FaceColor', [0.8500 0.3250 0.0980], '
    FaceAlpha', 0.7)
89 histogram(LDA_projection(1, I3), 'FaceColor', [0.9290 0.6940 0.1250], '
    FaceAlpha', 0.7)
90 title('Histogram of LDA 1');
91 hold off;
92
93 % Plot projections on LDA components 1 and 3
94 figure;
95 t = tiledlayout(4, 4, 'TileSpacing', 'Compact', 'Padding', 'Compact');
96
97 % Histogram for LDA component 3 (left y-axis)
98 ax1 = nexttile(1, [3, 1]);
99 histogram(LDA_projection(3, I1), 'FaceColor', [0 0.4470 0.7410], '
    FaceAlpha', 0.7)
100 hold on;
101 histogram(LDA_projection(3, I2), 'FaceColor', [0.8500 0.3250 0.0980], '
    FaceAlpha', 0.7)
102 histogram(LDA_projection(3, I3), 'FaceColor', [0.9290 0.6940 0.1250], '
    FaceAlpha', 0.7)
103 view(ax1, 90, -90)
104 title('Histogram of LDA 3');
105 hold off;
106
107 % Scatter plot of LDA components 1 and 3
108 ax2 = nexttile(2, [3, 3]);
109 scatter(LDA_projection(1, I1), LDA_projection(3, I1), 20, 'filled', "
    MarkerFaceColor", [0 0.4470 0.7410])
110 hold on;
111 scatter(LDA_projection(1, I2), LDA_projection(3, I2), 20, 'filled', "
    MarkerFaceColor", [0.8500 0.3250 0.0980])
112 scatter(LDA_projection(1, I3), LDA_projection(3, I3), 20, 'filled', "
    MarkerFaceColor", [0.9290 0.6940 0.1250])
113 xlabel('LDA Component 1', 'FontSize', 11);
114 ylabel('LDA Component 3', 'FontSize', 11);
115 legend('Setosa', 'Versicolor', 'Virginica');
116 hold off;
117 title('Scatter-plot of the the two-dimensional projection', 'FontSize',
    13)
118 subtitle('LDA 1 VS LDA 3', 'FontSize', 11.5)
119
120 % Histogram for LDA component 1 (below x-axis)
121 ax3 = nexttile(14, [1, 3]);
122 histogram(LDA_projection(1, I1), 'FaceColor', [0 0.4470 0.7410], '
    FaceAlpha', 0.7)
123 hold on;
124 histogram(LDA_projection(1, I2), 'FaceColor', [0.8500 0.3250 0.0980], '
    FaceAlpha', 0.7)

```

```

125 histogram(LDA_projection(1, I3), 'FaceColor', [0.9290 0.6940 0.1250], '
    FaceAlpha', 0.7)
126 title('Histogram of LDA 1');
127 hold off;
128
129 % Plot projections on LDA components 2 and 3
130 figure;
131 t = tiledlayout(4, 4, 'TileSpacing', 'Compact', 'Padding', 'Compact');
132
133 % Histogram for LDA component 3 (left y-axis)
134 ax1 = nexttile(1, [3, 1]);
135 histogram(LDA_projection(3, I1), 'FaceColor', [0 0.4470 0.7410], '
    FaceAlpha', 0.7)
136 hold on;
137 histogram(LDA_projection(3, I2), 'FaceColor', [0.8500 0.3250 0.0980], '
    FaceAlpha', 0.7)
138 histogram(LDA_projection(3, I3), 'FaceColor', [0.9290 0.6940 0.1250], '
    FaceAlpha', 0.7)
139 view(ax1, 90, -90)
140 title('Histogram of LDA 3');
141 hold off;
142
143 % Scatter plot of LDA components 2 and 3
144 ax2 = nexttile(2, [3, 3]);
145 scatter(LDA_projection(2, I1), LDA_projection(3, I1), 20, 'filled', "
    MarkerFaceColor", [0 0.4470 0.7410])
146 hold on;
147 scatter(LDA_projection(2, I2), LDA_projection(3, I2), 20, 'filled', "
    MarkerFaceColor", [0.8500 0.3250 0.0980])
148 scatter(LDA_projection(2, I3), LDA_projection(3, I3), 20, 'filled', "
    MarkerFaceColor", [0.9290 0.6940 0.1250])
149 xlabel('LDA Component 2', 'FontSize', 11);
150 ylabel('LDA Component 3', 'FontSize', 11);
151 legend('Setosa', 'Versicolor', 'Virginica');
152 hold off;
153 title('Scatter-plot of the the two-dimensional projection', 'FontSize',
    13)
154 subtitle('LDA 3 VS LDA 2', 'FontSize', 11.5)
155
156 % Histogram for LDA component 2 (below x-axis)
157 ax3 = nexttile(14, [1, 3]);
158 histogram(LDA_projection(2, I1), 'FaceColor', [0 0.4470 0.7410], '
    FaceAlpha', 0.7)
159 hold on;
160 histogram(LDA_projection(2, I2), 'FaceColor', [0.8500 0.3250 0.0980], '
    FaceAlpha', 0.7)
161 histogram(LDA_projection(2, I3), 'FaceColor', [0.9290 0.6940 0.1250], '
    FaceAlpha', 0.7)
162 title('Histogram of LDA 2');
163 hold off;
164
165 % Plotting the eigenvalues with and without regularization
166 figure;
167 eig_Sw = eig(Sw);
168 semilogy(eig_Sw, '*');
169 hold on;
170 semilogy(eig_Sw + epsilon, 'sm');
171 legend('eigenvalues', 'regularized eigenvalues');

```



```
172 title('Eigenvalue of Sw vs regularized eigenvalue', 'FontSize', 13);
173 xlabel('Index', 'FontSize', 11); % Adding x-axis label
174 ylabel('Eigenvalue', 'FontSize', 11); % Adding y-axis label
175 hold off;
176
177 % Check determinant before and after regularization
178 disp('det(Sw) before regularization');
179 det_Sw = det(Sw);
180 disp(det_Sw);
181
182 if (det_Sw <= 1e-14)
183     disp('Regularization applied');
184     Sw = Sw + epsilon * eye(size(Sw));
185     disp('det(Sw) after regularization');
186     det(Sw)
187 end
188
189 % Plotting eigenvalues before and after regularization
190 figure;
191 sSw = eig(Sw);
192 sSw_reg = eig(Sw + epsilon * eye(size(Sw)));
193 semilogy(sSw, 'r*');
194 hold on;
195 semilogy(sSw_reg, 'bd');
196 legend('eig before', 'eig after');
197 title('Eigenvalue of Sw before and after regularization', 'FontSize',
198       13);
199 xlabel('Index', 'FontSize', 11); % Adding x-axis label
200 ylabel('Eigenvalue', 'FontSize', 11); % Adding y-axis label
201 hold off;
```